

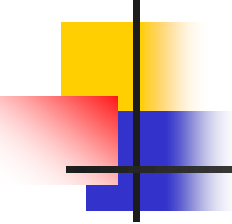
CS501

Introductory Computation for Scientists



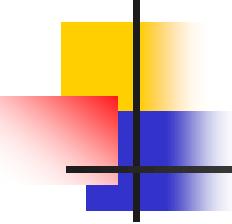
Dr. Ash Pahwa

Lesson 1: Programming Languages



Outline

- What is a Programming Language
- Types of Programming Languages
- History of Programming Languages
 - 1950 - 1960
 - 1960 - 1970
 - 1970 - 1980
 - 1980 - 1990
 - 1990 - 2000
 - 2000 - 2010
 - 2010 - Present
- Programming Languages for Data Science

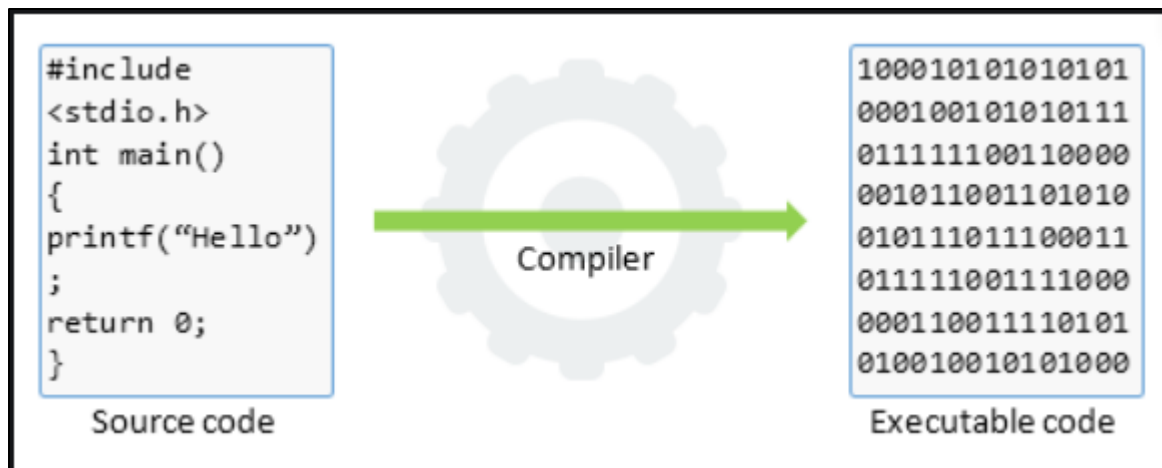


What is a Programming Language?

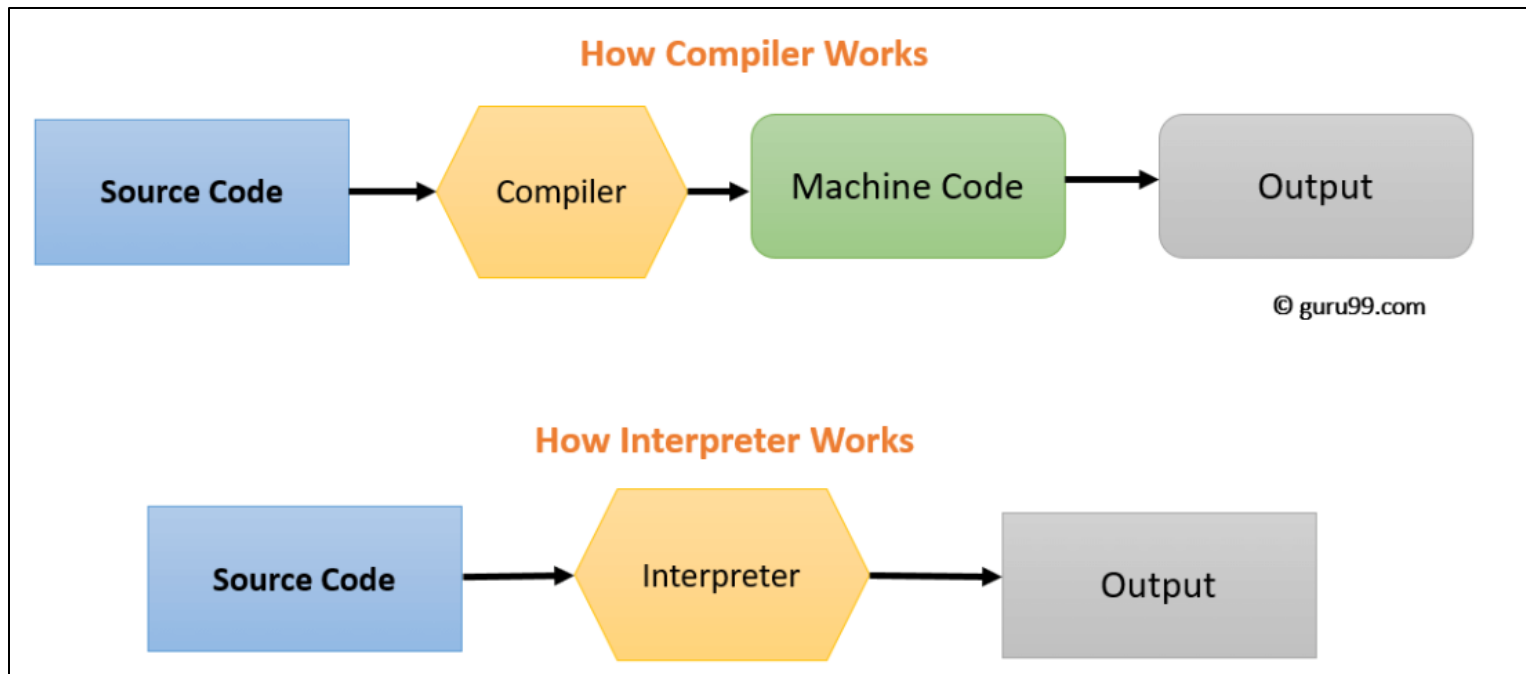
- A programming language is a Coded language used by programmers to write instructions
 - That a computer can understand to do what a programmer wants
- Compiler or interpreter converts the code into the machine language

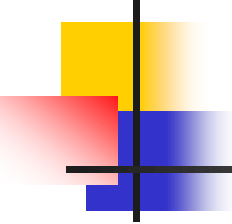
Compilation

- A Compiler converts high level language code into machine language



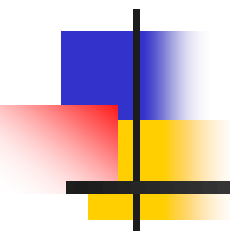
Compiler and Interpreter



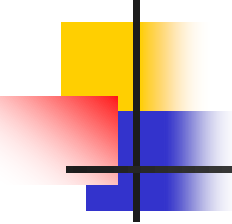


Difference Between Compiler and Interpreter

Compiler	Interpreter
Compiler will stop translating if it finds an error and there will be no executable code generated.	Interpreter will execute all the lines before error and will stop at the line which contains the error.
Compiler compile the program as whole.	It converts one statement of a program at one time.
Compiler generate object file.	Interpreter generates no object file.
Compiler translation is fast.	Interpreter translation is slow.



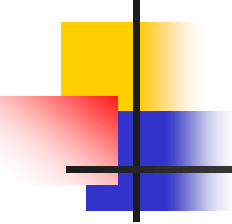
Types of Programming Languages



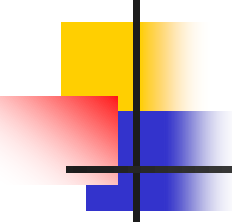
General Purpose Programming Language

- In computer software, a general-purpose programming language is a programming language designed to be used for writing software in the widest variety of application domains (a general-purpose language).
- A general-purpose programming language has this status because it does not include language constructs designed to be used within a specific application domain.

Object Oriented Programming Languages

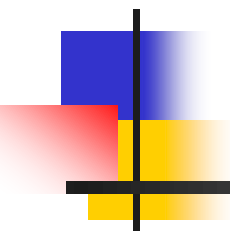


- Object-oriented programming (OOP) is a programming paradigm based on the concept of "objects", which can contain data, in the form of fields (often known as attributes or properties), and code, in the form of procedures (often known as methods).
- In OOP, computer programs are designed by making them out of objects that interact with one another.
- OOP languages are diverse, but the most popular ones are class-based, meaning that objects are instances of classes, which also determine their types.

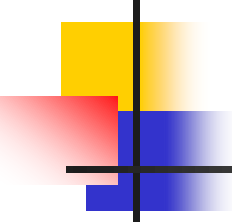


Functional Programming Language

- In computer science, functional programming is a programming paradigm—a style of building the structure and elements of computer programs—that treats computation as the evaluation of mathematical functions and avoids changing-state and mutable data.
- It is a declarative programming paradigm in that programming is done with expressions or declarations instead of statements.



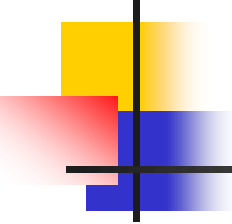
History of the Programming Languages



Programming Language

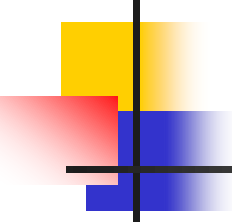
1950 - 1960

- Assembly Language
- Fortran: 1955 Formula Translation
- Lisp: 1958
- Algol: 1958
- Cobol: 1960 Common Business Oriented Language



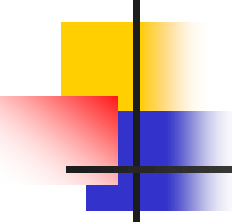
Fortran - 1955

- Fortran is a general-purpose programming language that is especially suited for numeric and scientific computation.
- Developed by IBM in 1954 and released in 1955.



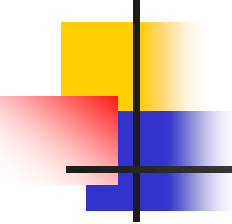
LISP (1958)

- LISP is a family of computer programming languages with a long history and a distinctive fully parenthesized prefix notation.
- LISP was originally created as a practical mathematical notation for computer programs



Cobol (1960)

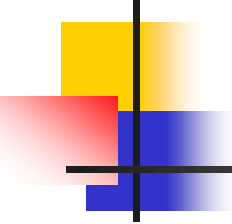
- COBOL an acronym for "common business-oriented language" is a compiled English-like computer programming language designed for business use.
- COBOL is primarily used in business, finance, and administrative systems for companies and governments.
- COBOL is still widely used in legacy applications deployed on mainframe computers, such as large-scale batch and transaction processing jobs.



Programming Language

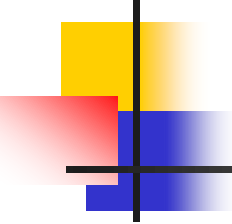
1960 - 1970

- Simula: 1962 – 1967
- Basic: 1964
- PL/1: 1964: IBM



BASIC (1964)

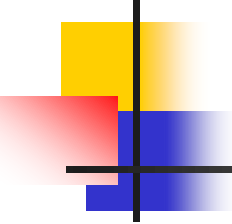
- Basic stands for Beginners All Purpose Symbolic Instruction Code.
- Adapted by Microsoft for its DOS and Windows operating system
- Invented by US math professors – John Kemeny and Thomas Kurtz of Dartmouth College



Programming Language

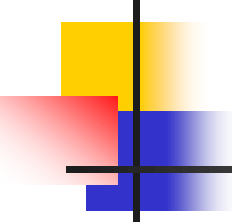
1970 - 1980

- Pascal: 1970
- C: 1972
- Modula: 1975
- SmallTalk: 1976
- C-Shell: 1978



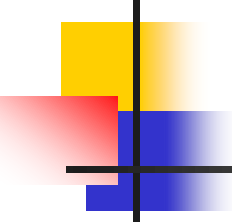
Pascal (1970)

- Pascal was developed by Niklaus Wirth in 1968 to 1969 and published in 1970
- Purpose: Encourage good programming practices using structured programming and data structuring.



C (1972)

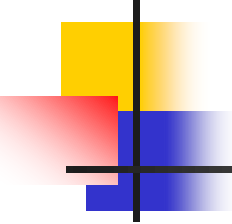
- C was developed by Dennis Ritchie in 1972 and it is a general-purpose computer programming language.
- C has since become one of the most widely used programming languages of all time, with C compilers from various vendors available for most existing computer architectures and operating systems.



Programming Language

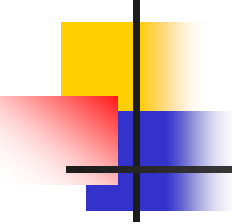
1980 - 1990

- Ada: Department of Defense (1980)
- Objective C: 1983
- C++ (1983)
- Perl (1987)



C++ (1983)

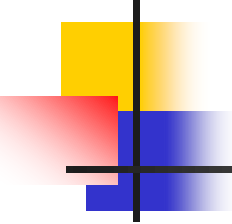
- C++ is an Object-Oriented programming language.
- It is a huge success to C, and currently the most used programming language on Mac, PC, Sun and other computer platforms.
- Developed by Bjarne Stroustrup of Bell Labs.



Programming Language

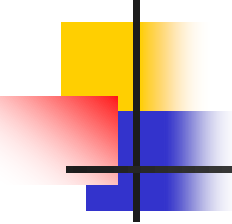
1990 - 2000

- Haskell (1990)
- Python (1991)
- Visual Basic (1991)
- Apple Script (1993)
- Java (1995)
- PHP (1995)
- Ruby (1995)
- JavaScript (1995)



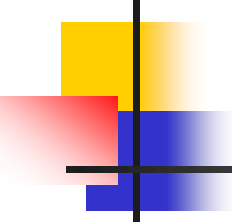
Python (1991)

- Python is a widely used high-level, general-purpose programming language
- Based on client-server architecture



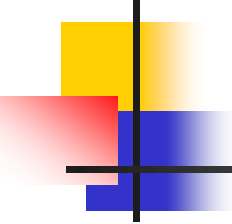
Java (1995)

- General Purpose versatile programming language that can run on virtually any type of computer
- Developed by James Gosling of Sun Microsystems specially for network heavy environment
- Java is Object Oriented language



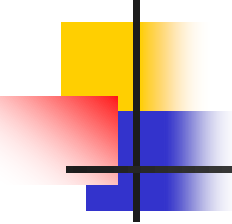
PHP (1995)

- PHP is a server-side scripting language designed for web development
- Originally created by Rasmus Lerdorf, PHP code can be embedded into HTML code
- PHP code is usually processed by a PHP interpreter implemented in a web server



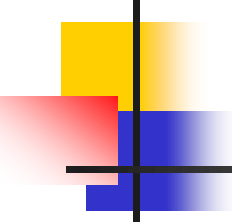
JavaScript (1995)

- JavaScript is a programming that enables novice programmers to easily design webpages and add interactive features
- Developed by Netscape (now a part of Times Warner) – not related to Sun Microsystems



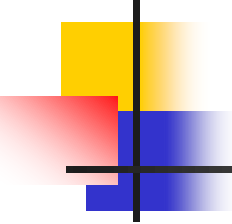
Programming Language 2000 - 2010

- ActionScript (2000)
- C# (2000): Microsoft
- ASP.NET (2000): Microsoft
- Visual Basic.Net (2001)
- Falcon (2003)
- Scala (2003)
- Windows PowerShell (2006)
- Go (2009): Google
- CoffeeScript (2009)



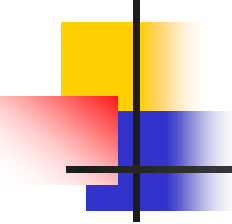
C# (2000)

- C# was developed by Anders Hejlsberg in 2000 and it is a multi-paradigm programming language
- It is Object-Oriented, Functional.
- It was developed by Microsoft within its .NET initiative.



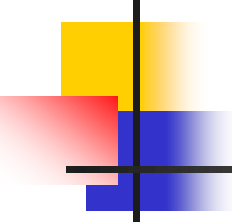
Go (2009)

- Go referred as golang, is an open source programming language created at Google in 2007 by Robert Griesemer, Rob Pike and Ken Thompson.
- It is used in some of Google's production systems.



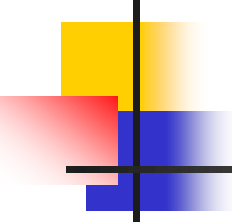
Programming Language 2010 -

- Dart (2011)
- C++11 (2011)
- TypeScript: 2012
- Swift: 2014
- C++14: 2014



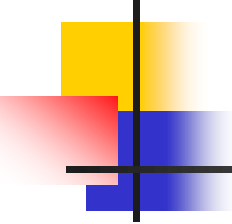
Dart (2011)

- Dart is a general-purpose programming language originally developed by Google and later approved as a standard by Ecma.
- It is used to build web servers and mobile applications.



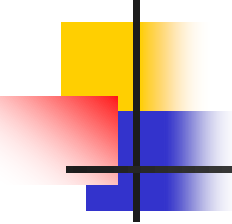
Swift (2014)

- Swift is a general-purpose, multi-paradigm, compiled programming language developed by Apple, Inc. for iOS, OSX and Linux.



Special Programming Languages

- SQL (1980)
 - Used to access data from a Relational Database
- Scientific Programming
 - MATLAB (1990)
 - R (2000)
 - Julia (2012)



Programming Languages for Data Science

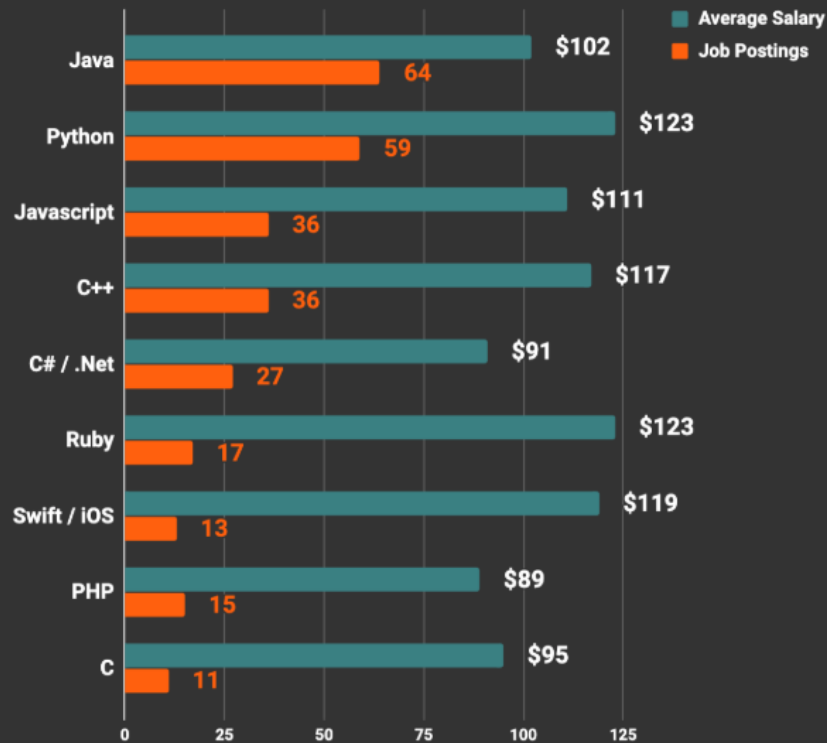
- Python (1991)
- MATLAB (1990)
- R (2000)
- Julia (2012)

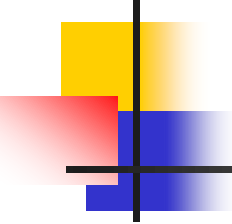
10 Most Popular Programming Languages

Ranking programming languages by pay and number of openings

Salary and job openings for popular programming languages, 2019

Figures, from Indeed, in thousands. Popularity based on TIOBE and StackOverflow indexes.





Summary

- What is a Programming Language
- Types of Programming Languages
- History of Programming Languages
 - 1950 - 1960
 - 1960 - 1970
 - 1970 - 1980
 - 1980 - 1990
 - 1990 - 2000
 - 2000 - 2010
 - 2010 - Present
- Programming Languages for Data Science

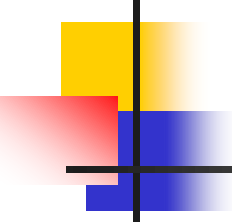
CS501

Introductory Computation for Scientists



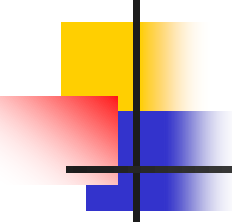
Dr. Ash Pahwa

Lesson 2.1: What is Python?



Outline

- What is Python?
- Python History
- Python Competitors
- Python Libraries
- Applications of Python
- Python2 and Python 3
- Advantages and Disadvantages of Python



What is Python?

- Python - History
- Open Source general purpose language created by Guido van Rossum in the late 1980s.

Guido van Rossum

- Guido van Rossum is a Dutch programmer who is best known as the author of the Python programming language.
- He was employed by Google from 2005 until 7 December 2012, where he spent half his time developing the Python language.
- In January 2013, Van Rossum started working for Dropbox.



Guido van Rossum

Programmer

Guido van Rossum is a Dutch programmer who is best known as the author of the Python programming language. [Wikipedia](#)

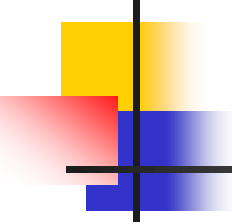
Born: January 31, 1956 (age 61), Haarlem, Netherlands

Education: University of Amsterdam (1982)

Spouse: Kim Knapp (m. 2000)

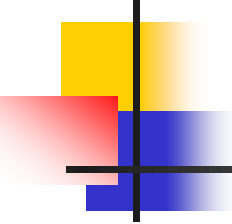
Awards: Award for Advancement of Free Software

Siblings: [Just van Rossum](#)



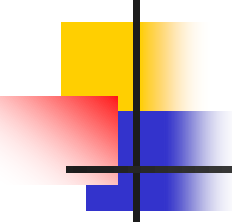
A Few Famous Software Programs Written In Python?

- YouTube
- Dropbox
- Google
- Instagram
- Spotify
- Reddit
- Yahoo Maps



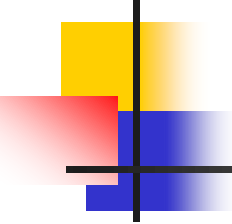
Python Competitors





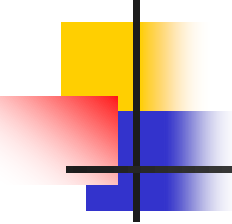
Python Libraries

- NumPy
 - Providing a practical array of data structures
- SciPy
 - Scientific Computing
- Matplotlib
 - Graphical plotting
- Pandas
 - Data analysis and statistics
- Scikit-learn
 - Machine Learning
- SymPy
 - Symbolic Computing
- Jupyter
 - Interactive Computing



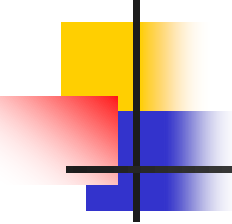
Other Python Applications

- Web Development
 - Django, Pyramid, Flask
- Computer Vision
- Raspberry Pi
 - Robotics
- Game Development
 - Pygame
- Web Scraping
- Writing Scripts
- Browser Automation
- GUI Development



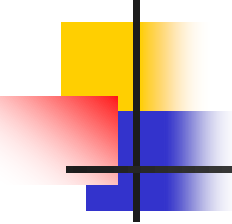
Python 2 or Python 3

- Python 2.0 was first released in 2000
 - Its latest version, 2.7, was released in 2010
- Python 3.0 was released in 2008
 - Its latest version is 3.7
- Python 2 code will not run on Python 3



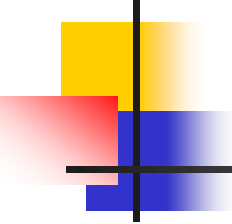
Advantages of Python

- Simple syntax: Easy to learn
- It is free: Open Source
- Cross Platform support: Windows + Mac + Linux
- Rich set of libraries (modules): Numpy, Scipy, ... etc.
- Easy to learn
- Use heavily in
 - Data Science
 - Machine Learning
 - Scientific Computing
- Python: Contains features of
 - Procedural languages (General Purpose)
 - Object Oriented Languages
 - Functional Languages



Disadvantages of Python

- Interpreted Language: Not as fast as compiled language
- Python 2 and Python 3 problems



Summary

- What is Python?
- Python History
- Python Competitors
- Python Libraries
- Applications of Python
- Python2 and Python 3
- Advantages and Disadvantages of Python

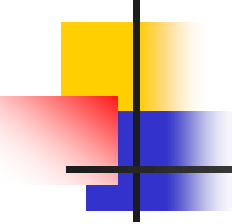
CS501

Introductory Computation for Scientists



Dr. Ash Pahwa

Lesson 2.2: Anaconda + Jupyter + Spyder

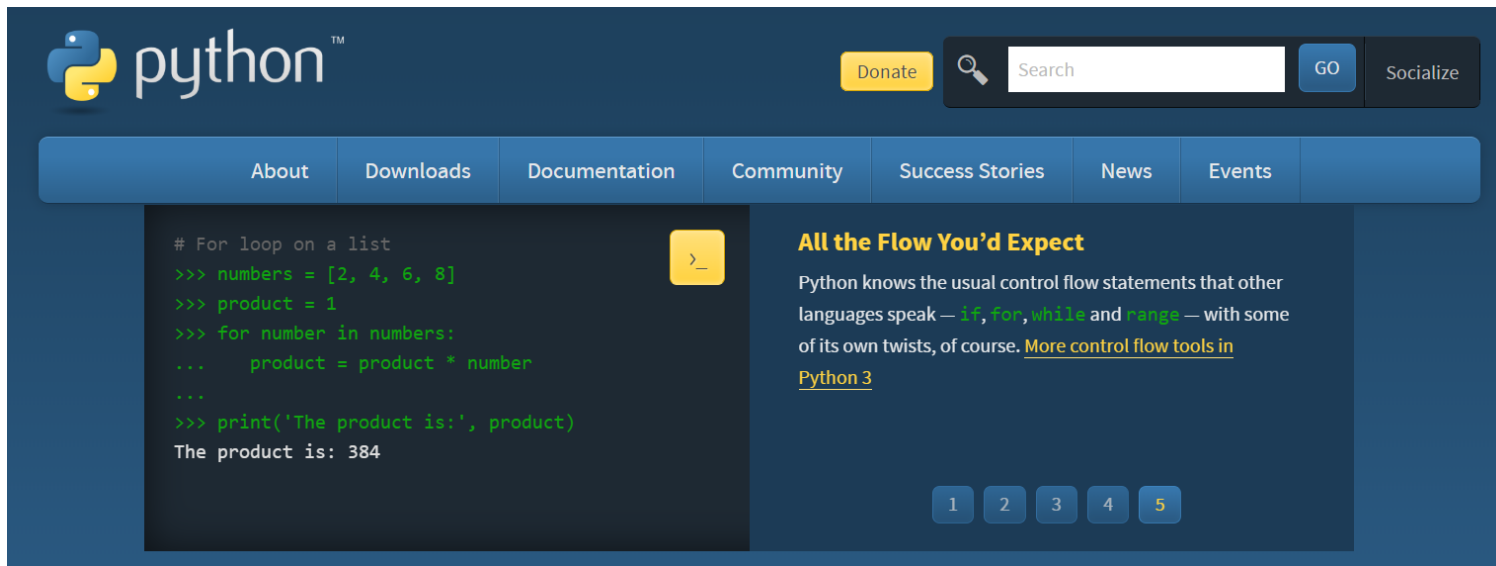


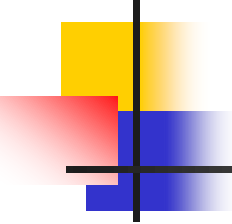
Outline

- Installing Anaconda – Download Software
- Running Anaconda
- IDE – Integrated Development Environment
 - IDLE
 - Jupyter Notebook
 - Running Jupyter
 - What is Spyder
 - Running Spyder

Installing Python

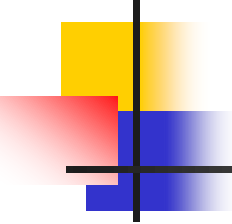
- www.python.org





Full Distributions From Anaconda

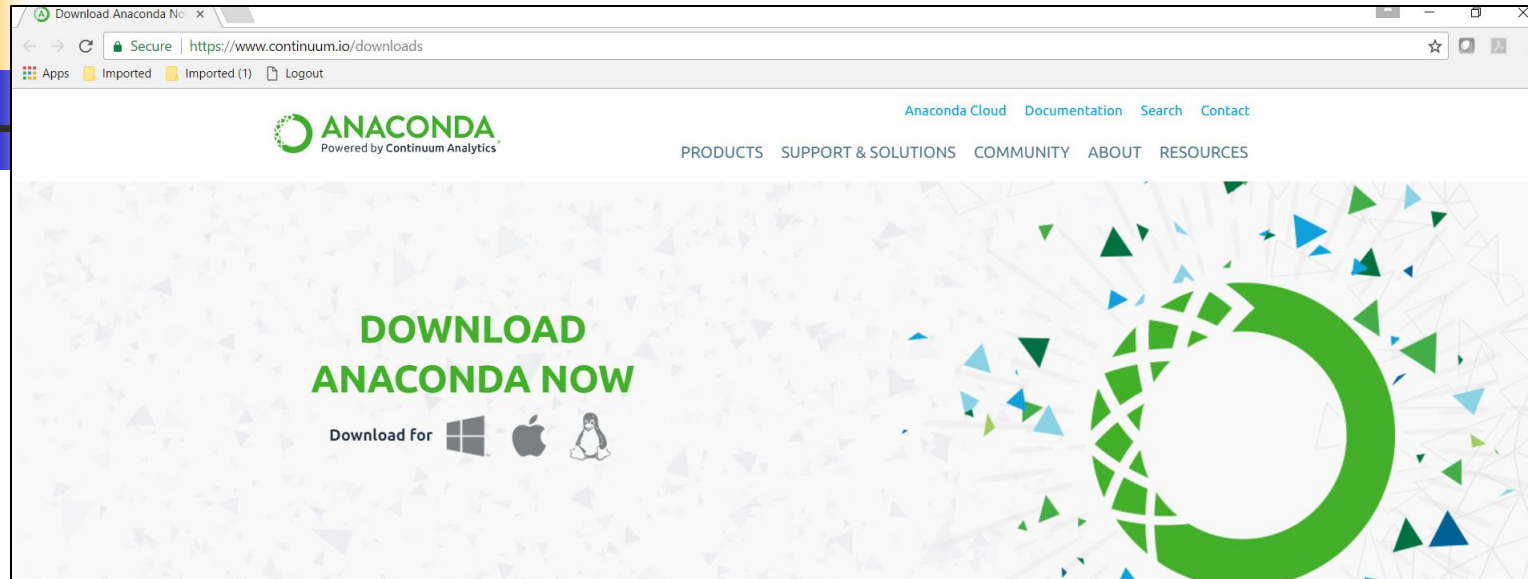
- Anaconda is a free open source distribution of the Python programming languages
- Application: For large-scale data processing, predictive analytics, and scientific computing
- Purpose: Anaconda aims to simplify package management and deployment



Anaconda Distribution

- ANACONDA PACKAGE LIST
 - Anaconda 4.3.1 includes installation of
 - Python (2.7.13, 3.4.5, 3.5.2, and/or 3.6.0) and
 - Updates of over 100 pre-built and tested scientific and analytic Python packages.
- These packages include the following
 - NumPy
 - Pandas
 - SciPy
 - Matplotlib
 - Jupyter
 - Scikit-Learn

<https://continuum.io/downloads>



[Download for Windows](#)[Download for macOS](#)[Download for Linux](#)

Anaconda 4.3.1

For Windows

Anaconda is BSD licensed which gives you permission to use Anaconda commercially and for redistribution.

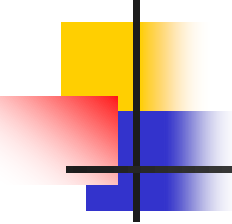
Python 3.6 version

64-BIT INSTALLER (422M)

[32-BIT INSTALLER \(348M\)](#)

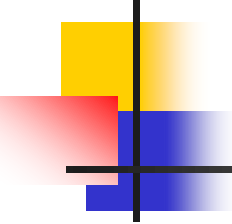
Run the Anaconda3-4.3.1-Windows-x86_64.exe file





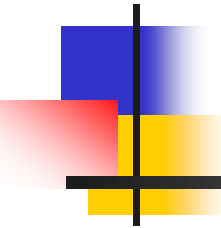
Package Manager - pip

- pip is a package management system used to install and manage software packages written in Python.
- pip is a recursive acronym that can stand for either "Pip Installs Packages" or "Pip Installs Python"



Advantage of Anaconda

- Anaconda comes with package manager 'Conda'
- 'Conda' is similar to pip.
- 'Conda' allows you to install non-python libraries dependencies such as
 - HDF5, MKL, LLVM etc.
- With 'pip' you can only install Python dependencies.



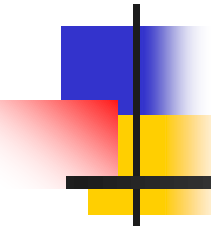
Running Anaconda

Command Prompt

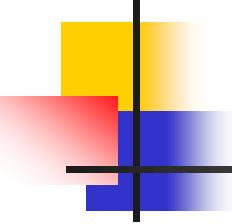
'conda list'

```
Anaconda Prompt

(C:\Users\ash\Anaconda3) C:\Users\ash>conda list
# packages in environment at C:\Users\ash\Anaconda3:
#
_license                  1.1                      py36_1
alabaster                  0.7.9                   py36_0
anaconda                   4.3.0                   np111py36_0
anaconda-client            1.6.0                   py36_0
anaconda-navigator         1.4.3                   py36_0
astroid                    1.4.9                   py36_0
astropy                    1.3                     np111py36_0
babel                      2.3.4                   py36_0
backports                  1.0                     py36_0
beautifulsoup4             4.5.3                   py36_0
bitarray                   0.8.1                   py36_1
blaze                      0.10.1                  py36_0
bokeh                      0.12.4                  py36_0
boto                       2.45.0                  py36_0
bottleneck                 1.2.0                   np111py36_0
bzip2                     1.0.6                   vc14_3 [vc14]
cffi                       1.9.1                   py36_0
chardet                    2.3.0                   py36_0
chest                      0.2.3                   py36_0
click                      6.7                     py36_0
cloudpickle                0.2.2                   py36_0
clyent                     1.2.2                   py36_0
colorama                   0.3.7                   py36_0
comtypes                   1.1.2                   py36_0
conda                      4.3.8                   py36_0
conda-env                  2.6.0                   0
```



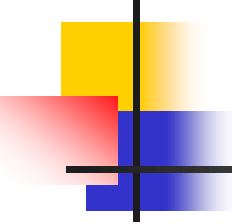
Running Python



Running Python

Integrated Development Environment (IDE)

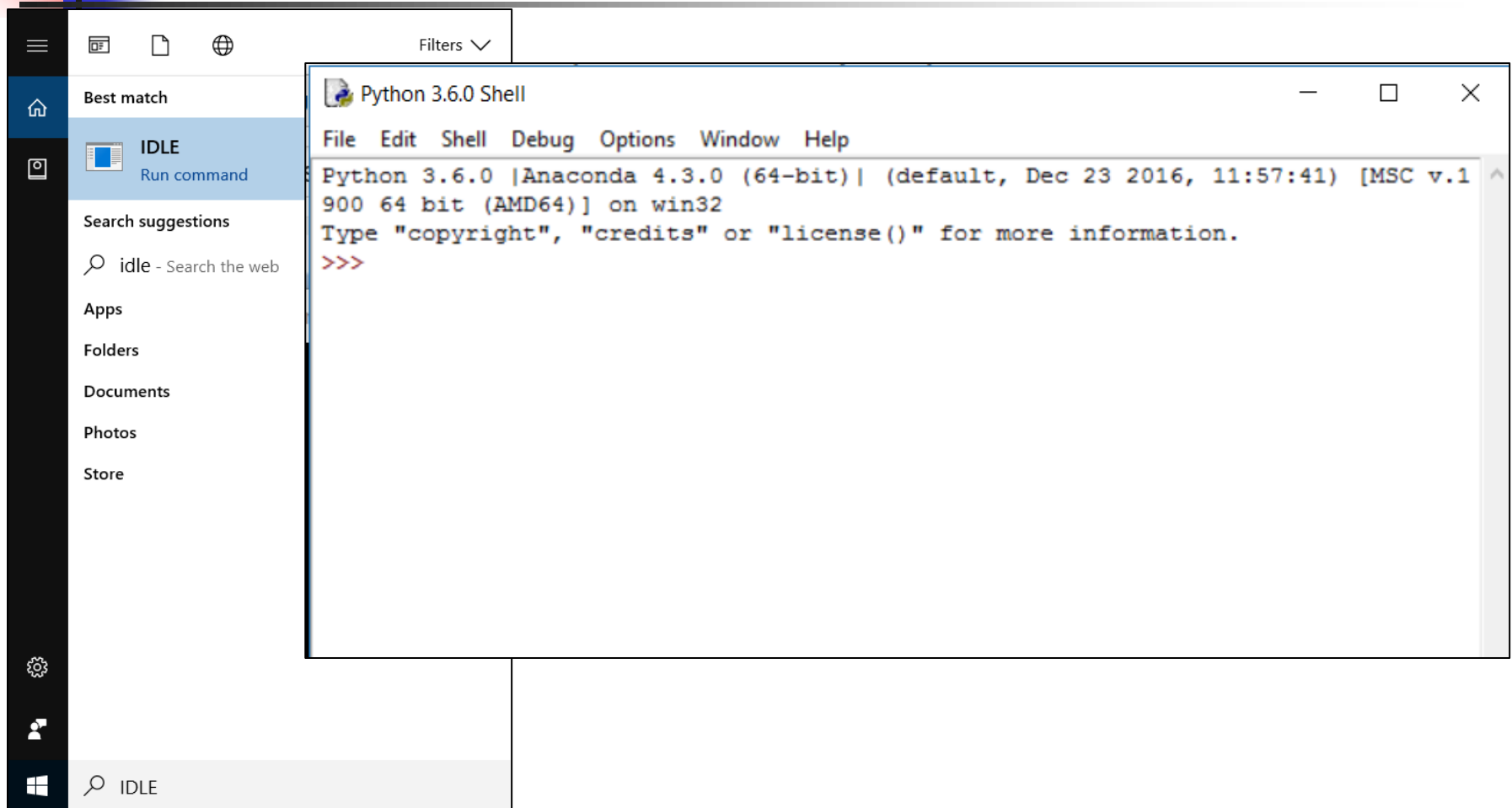
- IDLE
- Jupyter Notebook
 - Interactive Document
- Spyder
 - Native Python Code
- PyCharm
- Rodeo
- Atom

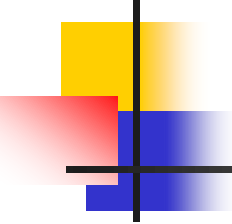


To Run Python Execute: IDLE

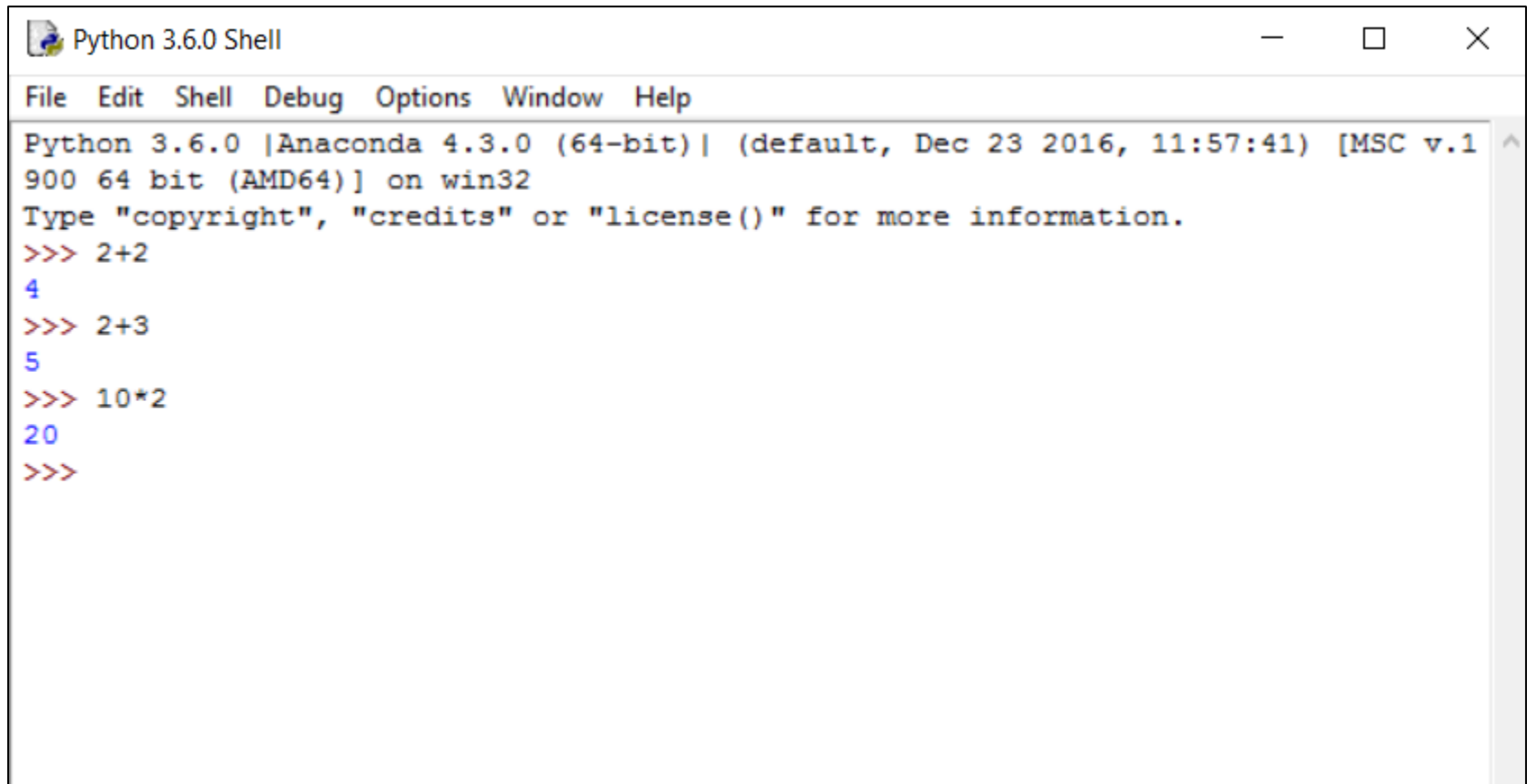
- IDLE
 - Integrated DeveLopment Environment or
 - Integrated Development and Learning Environment
 - Is an integrated development environment for Python, which has been bundled with the default implementation of the language

To Run Python Execute: IDLE



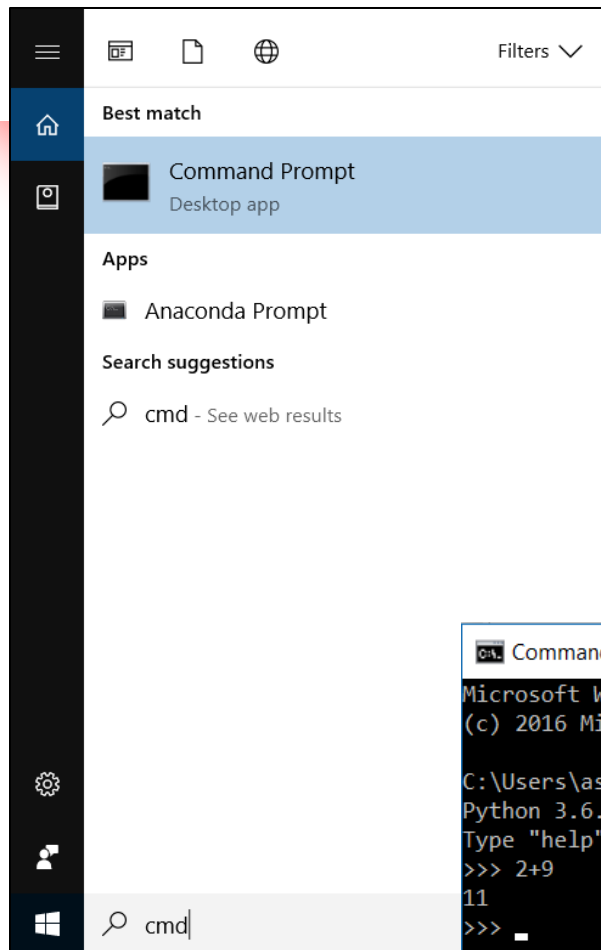


A Simple Session using Python



```
Python 3.6.0 Shell
File Edit Shell Debug Options Window Help
Python 3.6.0 [Anaconda 4.3.0 (64-bit)] (default, Dec 23 2016, 11:57:41) [MSC v.1
900 64 bit (AMD64)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> 2+2
4
>>> 2+3
5
>>> 10*2
20
>>>
```

Command Prompt: cmd + python



Command Prompt

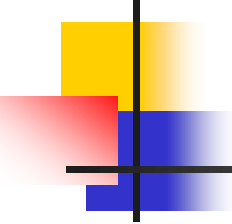
```
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\Users\ash>python_
```

Command Prompt - python

```
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

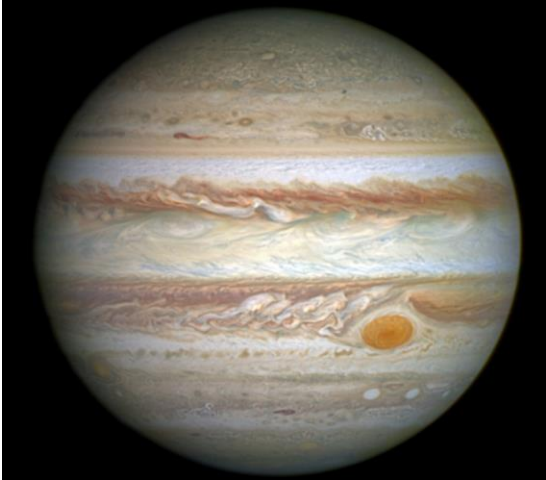
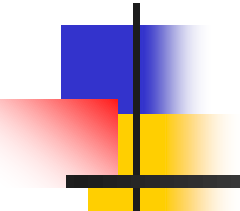
C:\Users\ash>python
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [MSC v.1900 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> 2+9
11
>>> _
```



IDLE vs cmd+python

- Use IDLE instead of cmd+python
- IDLE provides color fonts
- IDLE provides many other features that cmd+python doesn't provide

Jupyter Notebook

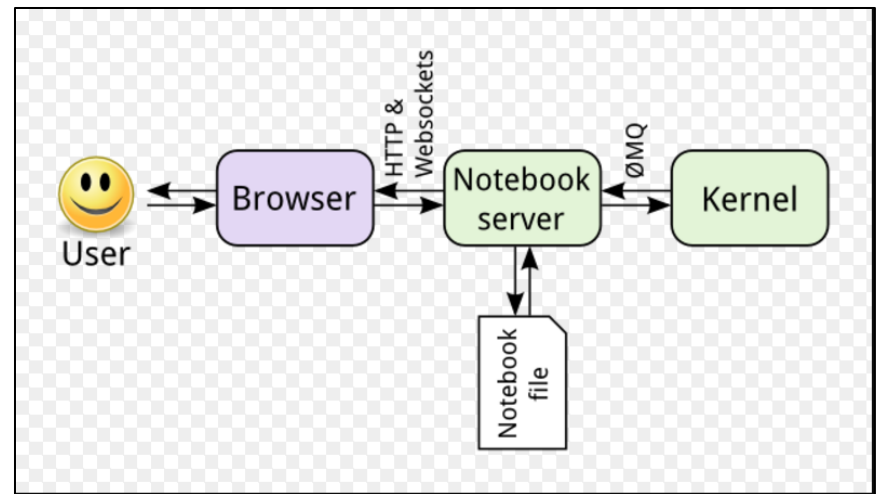


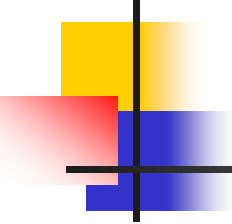
Jupiter Planet



Jupyter Architecture

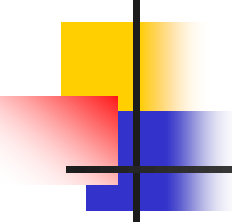
- User Interacts with Browser
- Browser communicates with Notebook Server
- Server uses Kernel for a specific language
- Jupyter commands are stored in a file





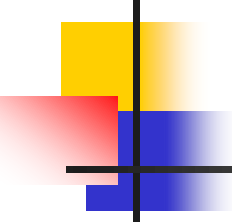
What is Jupyter?

- Jupyter is a language independent interactive development environment for
 - Scientific Computing
 - Data Science
- It optimizes the workflow for modern
 - Scientific Computing and
 - Data Science



Jupyter Components

- Web Application
- Kernels
 - Based on a specific language (Python is one of the languages)
- Notebook document
 - Code (Python)
 - Text Annotation (HTML)
 - Images/Video
 - Each document is stored in JSON format



JSON

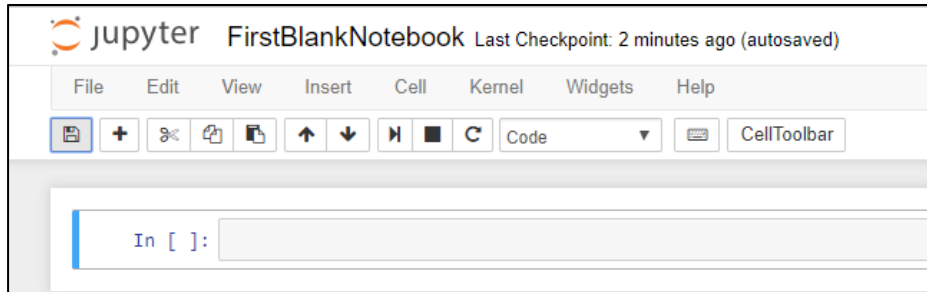
- File Format
 - Jupyter: JSON
 - Spyder: ASCII
- **JSON** (JavaScript Object Notation) is a lightweight data-interchange format. It is easy for humans to read and write. It is easy for machines to parse and generate.
- It is used primarily to transmit data between a server and web application, as an alternative to **XML**.

JSON

MS WORD can Open JSON File

File Name:

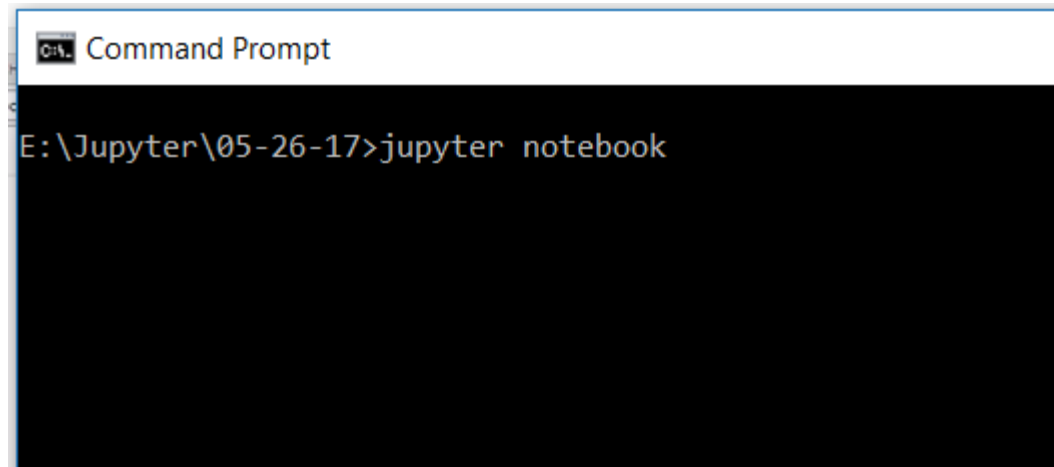
FirstBlankNotebook.ipynb



```
{
  "cells": [
    {
      "cell_type": "code",
      "execution_count": null,
      "metadata": {
        "collapsed": true
      },
      "outputs": [],
      "source": []
    }
  ],
  "metadata": {
    "kernelspec": {
      "display_name": "Python 3",
      "language": "python",
      "name": "python3"
    },
    "language_info": {
      "codemirror_mode": {
        "name": "ipython",
        "version": 3
      },
      "file_extension": ".py",
      "mimetype": "text/x-python",
      "name": "python",
      "nbconvert_exporter": "python",
      "pygments_lexer": "ipython3",
      "version": "3.6.0"
    }
  },
  "nbformat": 4,
  "nbformat_minor": 2
}
```

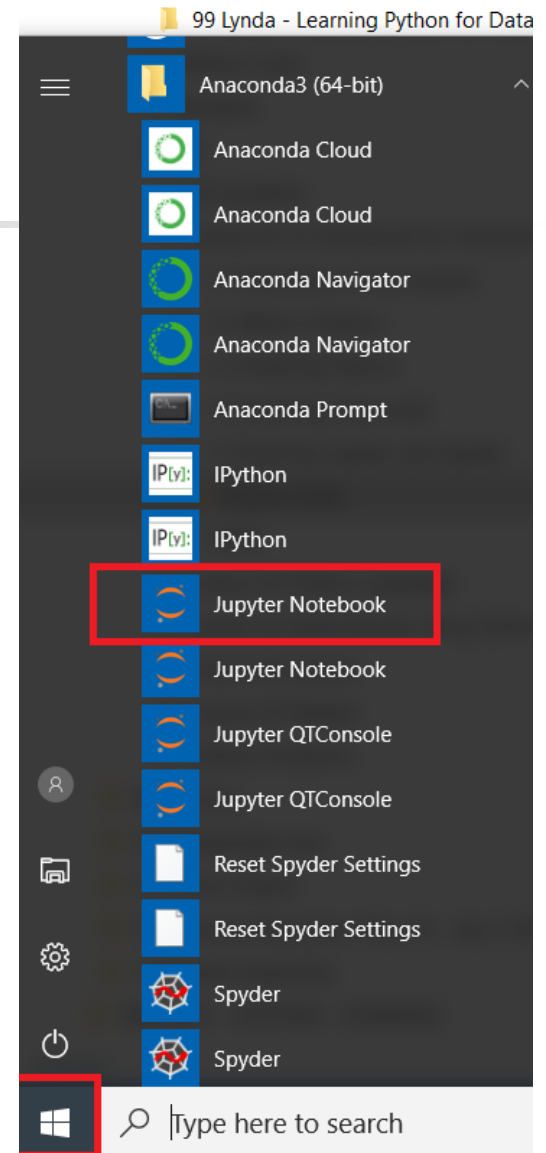
Launch Command Prompt

- Windows Command prompt
- Enter command
 - Jupyter notebook



Start Jupyter

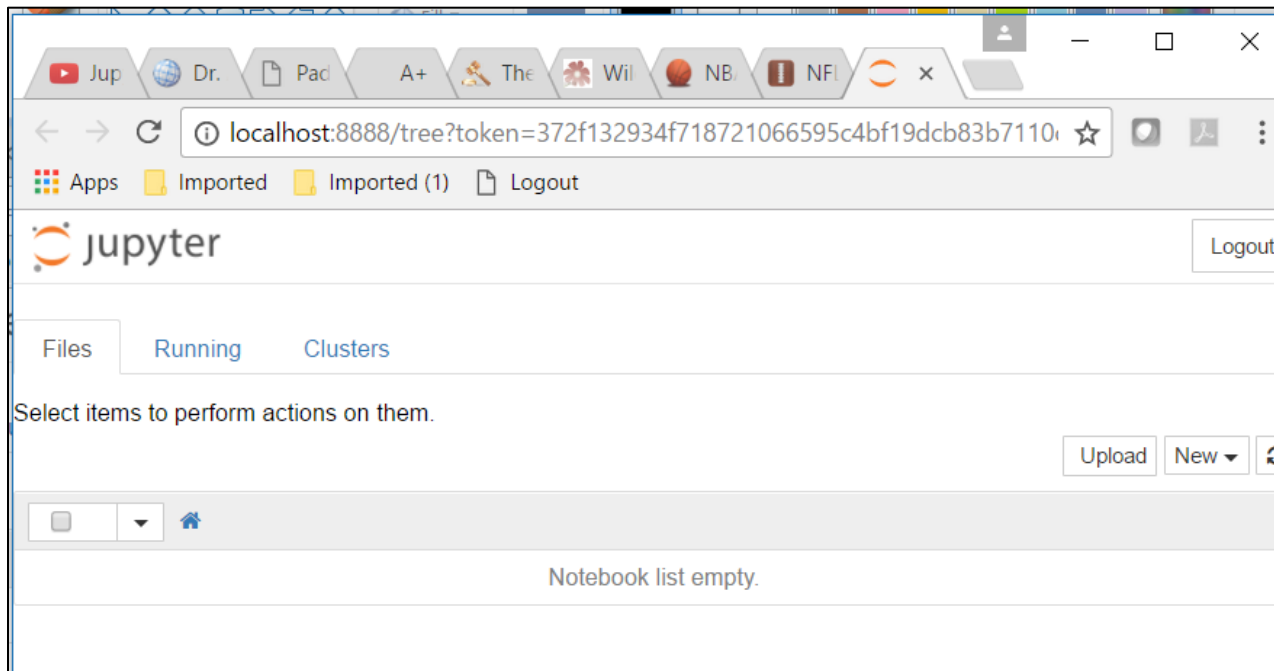
- Start Button
Anaconda3/Jupyter Notebook
- Jupyter will run in your Browser



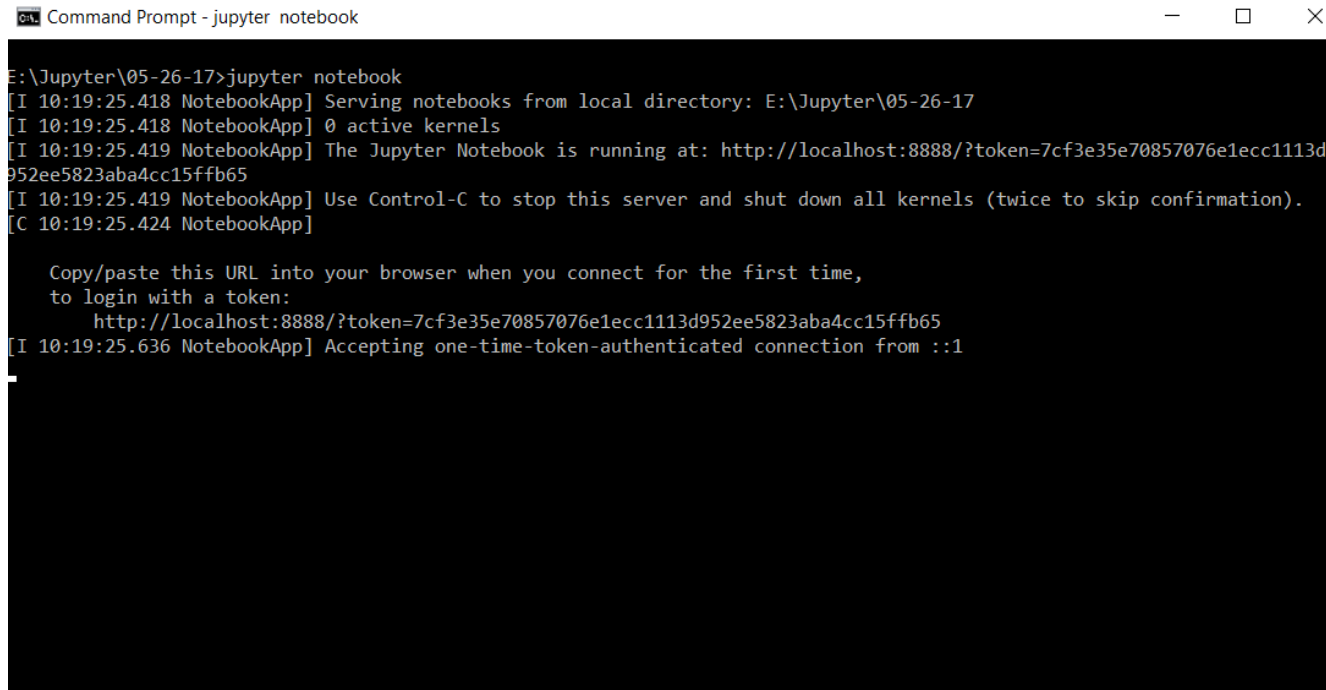
Dashboard

Jupyter will run in Browser

- File: To explorer to see which files are available
- Running: Displays Jupyter processes
- Clusters: Manage parallel computing
- Upload: Select files from your computer
- New: Create new notebooks



Notebook Server Runs in Command Prompt



```
Command Prompt - jupyter notebook

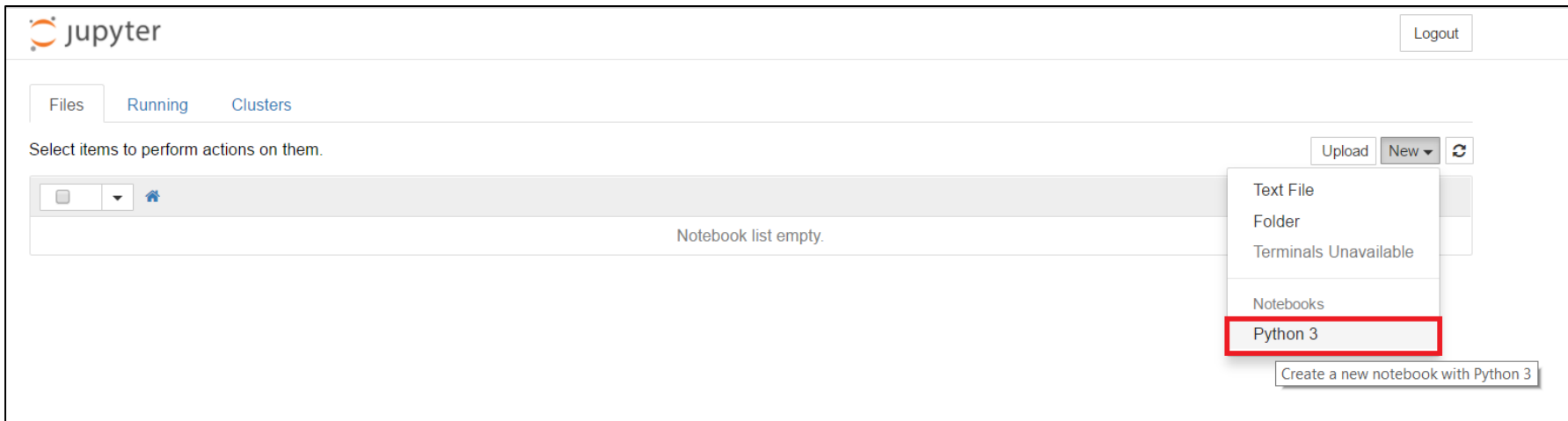
E:\Jupyter\05-26-17>jupyter notebook
[I 10:19:25.418 NotebookApp] Serving notebooks from local directory: E:\Jupyter\05-26-17
[I 10:19:25.418 NotebookApp] 0 active kernels
[I 10:19:25.419 NotebookApp] The Jupyter Notebook is running at: http://localhost:8888/?token=7cf3e35e70857076e1ecc1113d952ee5823aba4cc15ffb65
[I 10:19:25.419 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[C 10:19:25.424 NotebookApp]

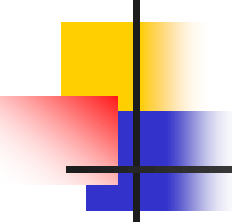
Copy/paste this URL into your browser when you connect for the first time,
to login with a token:
    http://localhost:8888/?token=7cf3e35e70857076e1ecc1113d952ee5823aba4cc15ffb65
[I 10:19:25.636 NotebookApp] Accepting one-time-token-authenticated connection from ::1
```

Dashboard

Create a new Python 3 Notebook

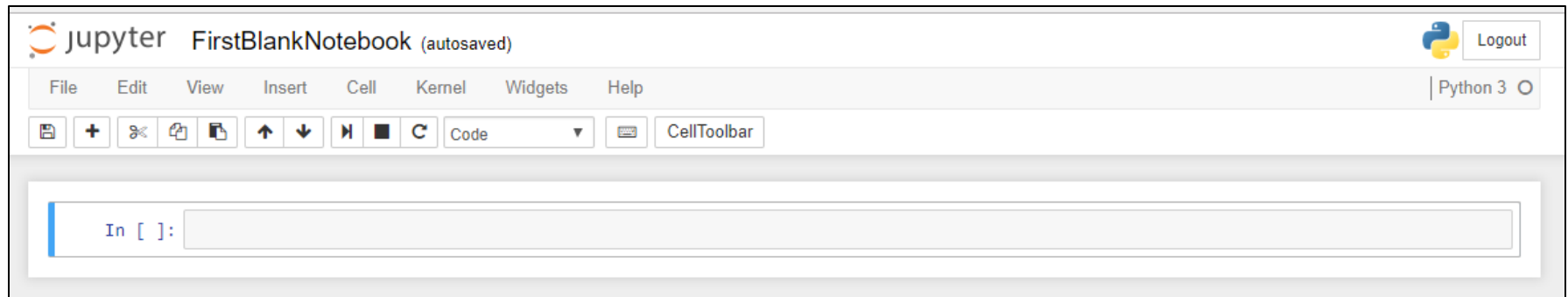
- Dashboard: New/Python 3





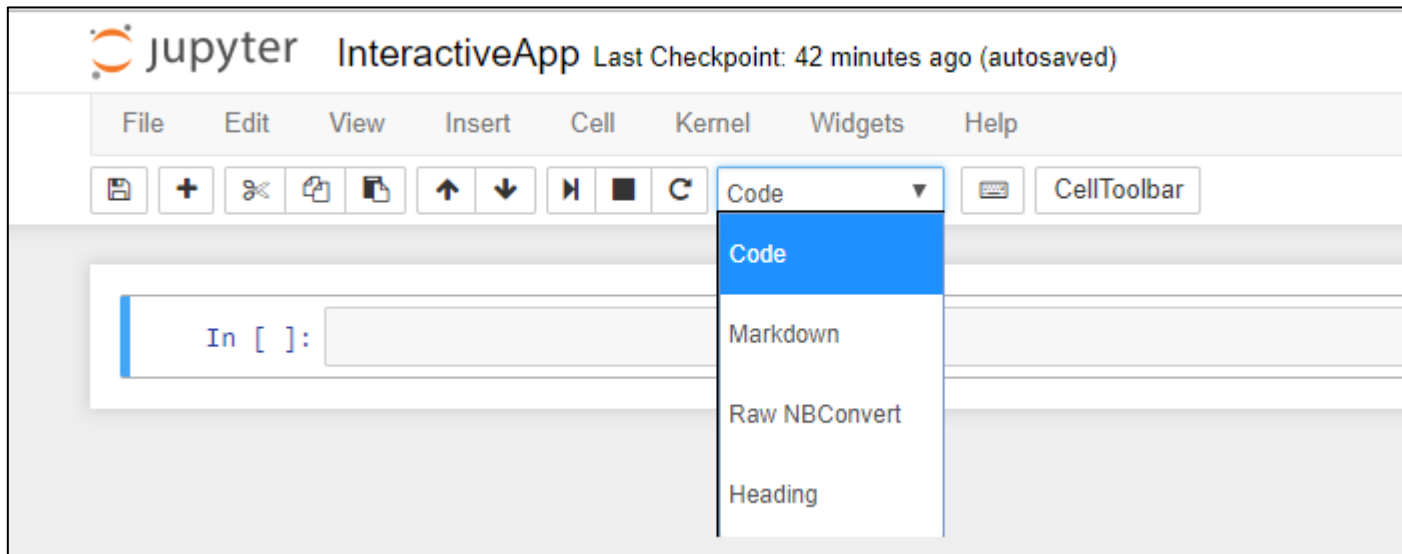
Jupyter: Header Notebook

- First Blank Notebook
 - Header
 - Title
 - Menu bar
 - Tool bar

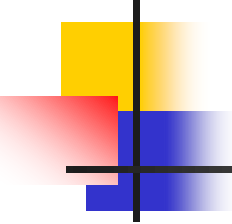


Jupyter: Body Notebook

- Body: Cells
 - Markdown: HTML
 - Code: Python Code
 - Raw Text







What is Spyder?

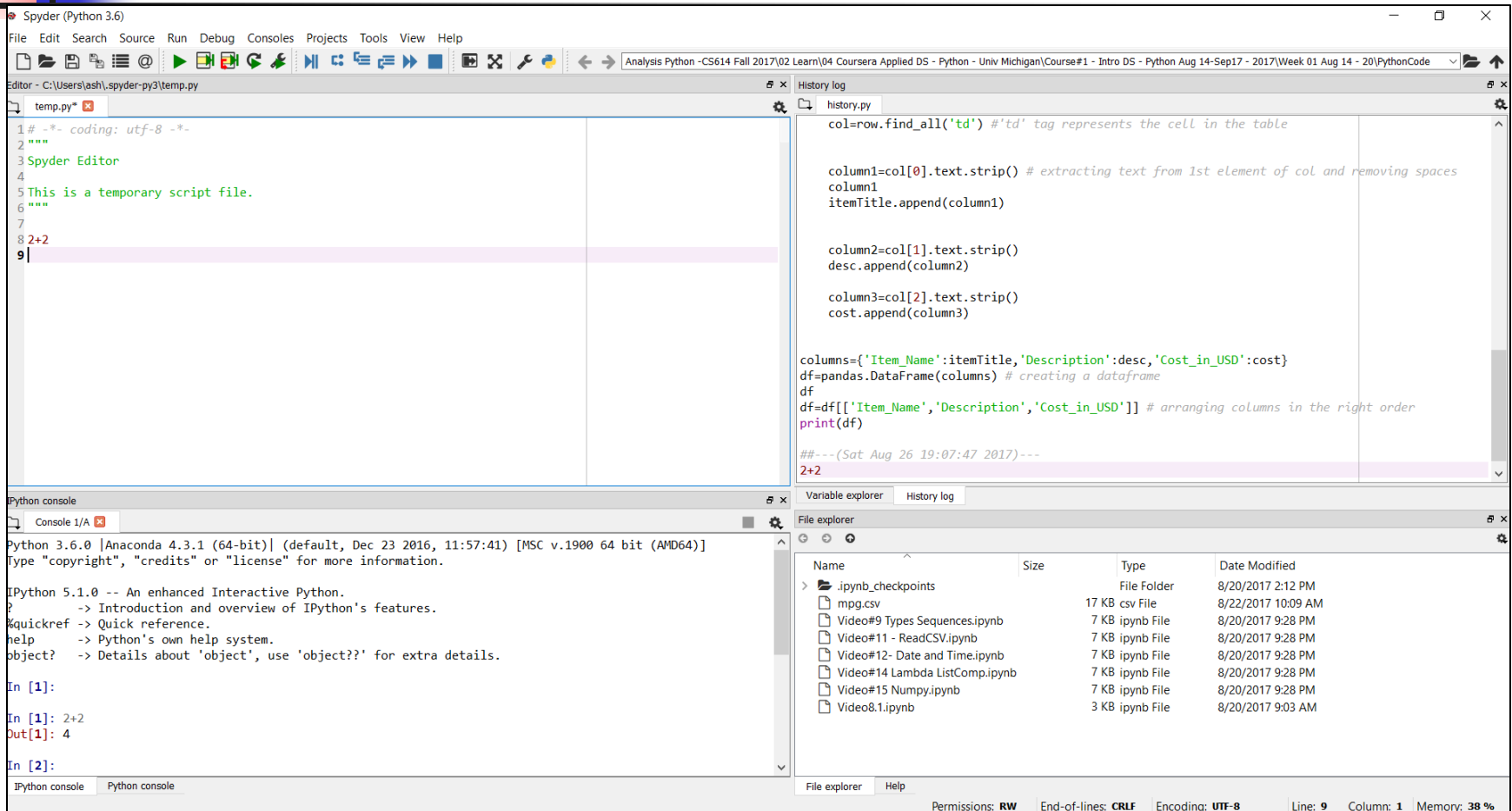
- Spyder (formerly Pydee) is an open source cross-platform integrated development environment (IDE) for scientific programming in the Python language.
- Spyder integrates NumPy, SciPy, Matplotlib and IPython, as well as other open source software.

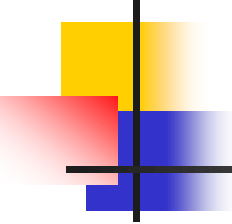
What is Spyder?

- Spyder Logo



Spyder GUI : Similar to RStudio





Summary

- Installing Anaconda – Download Software
- Running Anaconda
- IDE – Integrated Development Environment
 - IDLE
 - Jupyter Notebook
 - Running Jupyter
 - What is Spyder
 - Running Spyder

CS501

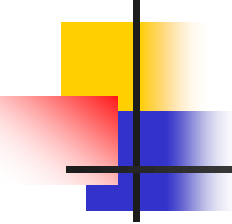
Introductory Computation for Scientists



Dr. Ash Pahwa

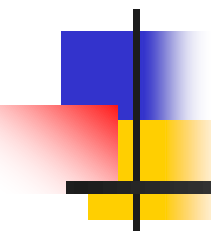
Lesson 2.3

Google's Colab

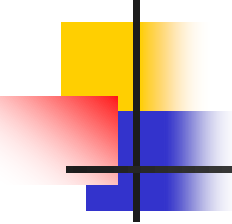


Outline

- Colab
 - What is Colab?
 - Cost of Colab
 - Benefits and Drawbacks of Colab
 - How to Use Colab
 - Data IO in Colab

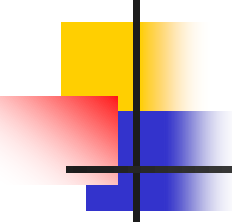


Google's Colab



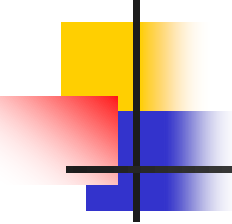
What is Colab?

- Colaboratory or “Colab” is a Google research project created to help disseminate machine learning education and research
- It's a Jupyter notebook environment that requires no setup to use and runs entirely in the cloud



When did Google Colab come out? 2017

- **Google** first started working with the Jupyter Development Team in 2014 to **release** an early version of the tool
- Google Colab is identical in Jupyter Notebook
- Google have released Colaboratory: a web IDE for python, to enable Machine Learning with storage on the cloud
- This internal tool had a quiet public release in late 2017 and is set to make a huge difference in the world of machine learning, artificial intelligence and data science work

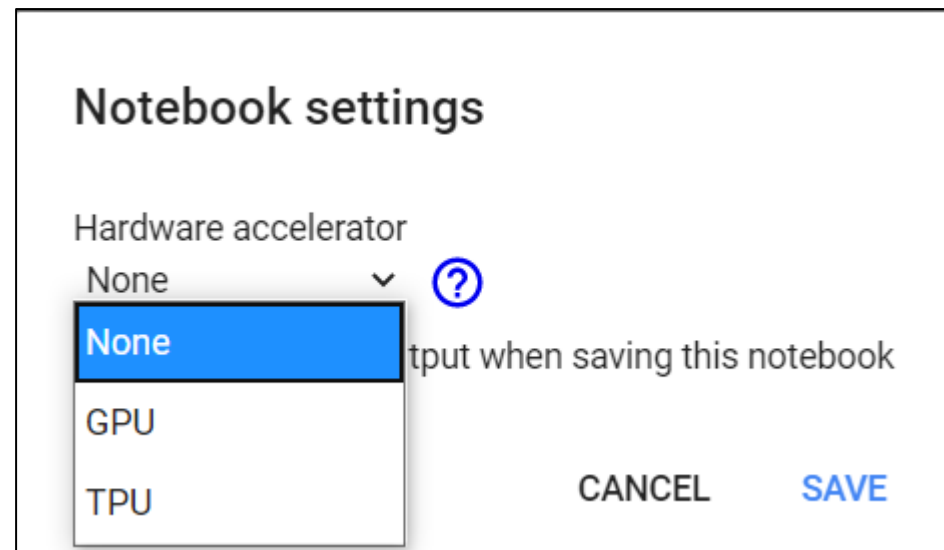
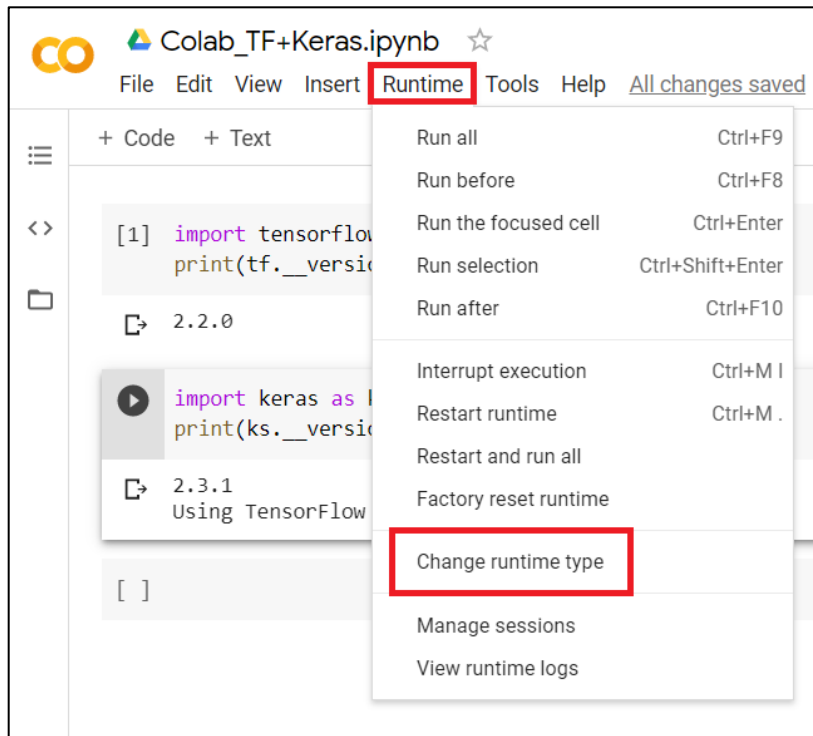


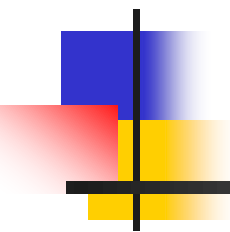
Cost of Using Colab

- Google Colab is a **free** to use Jupyter notebook
- Allows you to use **free** Tesla K80 GPU
- RAM of 12 GB
- You can use Colab
 - 24 hours continuously without GPU
 - 12 hours with a GPU

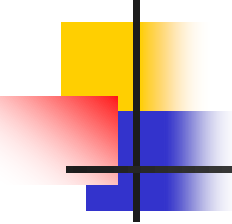
GPU Access

- Runtime > Change runtime type
- Select GPU as Hardware accelerator



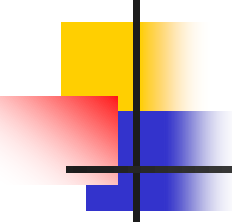


Benefits and Drawbacks of Google's Colab



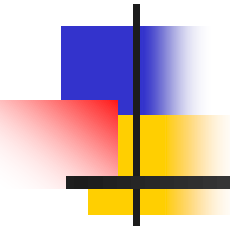
Benefits of Colab

- **Free** virtual machines for you to use with about 12GB RAM and 50GB hard drive space, with common dependencies such as numpy, pandas, and even TensorFlow pre-installed.
- **Free** GPU access
- Supports Python 2 and Python 3
- There is integration with Google Drive, you can share, and control permissions and you'll be able to see other collaborators work instantly
- You can import an existing Jupyter/IPython notebooks



Drawbacks of Colab

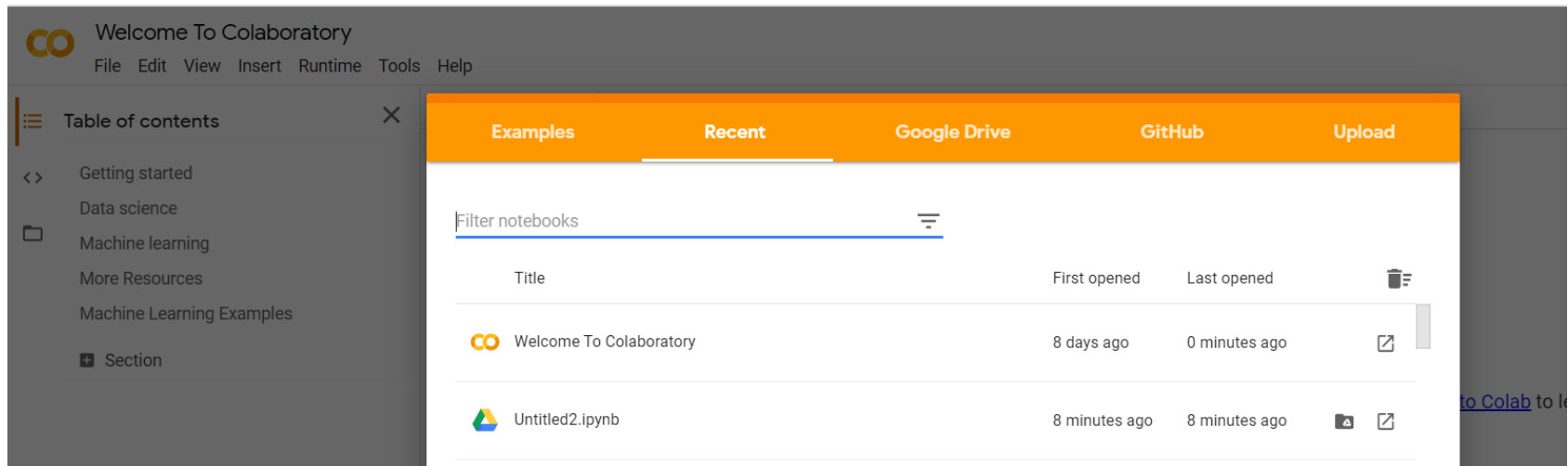
- All Colab notebooks must be stored in Google Drive — so you'll need to log into a Google account before you can access the tool
- Long-running background computations may be stopped
- You'll need to install all specific libraries which do not come with standard python (and you'll need to repeat this with every session)
- Google Storage is used with your current session, so if you have downloaded a file and want to use it later, you'd better save it before closing the session
- It can be difficult (and potentially costly) to work with bigger datasets as you must download and store them in Google drive (only **15GB** is free in Google Drive)



How do I use Google Colab?

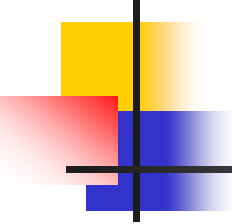
How do I use Google Colab?

- To start working with **Colab** you first need to log in to your **google** account, then go to this link
- <https://colab.research.google.com>.
 - **EXAMPLES**: Contain a number of Jupyter notebooks of various examples.
 - **RECENT**: Jupyter notebook you have recently worked with.
 - **GOOGLE DRIVE**: Jupyter notebook in your **google** drive.



The screenshot shows the Google Colaboratory web interface. On the left is a sidebar with a 'Table of contents' panel listing 'Getting started', 'Data science', 'Machine learning', 'More Resources', and 'Machine Learning Examples'. The main area has a top navigation bar with 'Examples', 'Recent', 'Google Drive', 'GitHub', and 'Upload' tabs. The 'Recent' tab is active, displaying a table of notebooks. The table has columns for 'Title', 'First opened', 'Last opened', and an action column with icons for deleting and sharing. Two notebooks are listed: 'Welcome To Colaboratory' (opened 8 days ago, 0 minutes ago) and 'Untitled2.ipynb' (opened 8 minutes ago, 8 minutes ago).

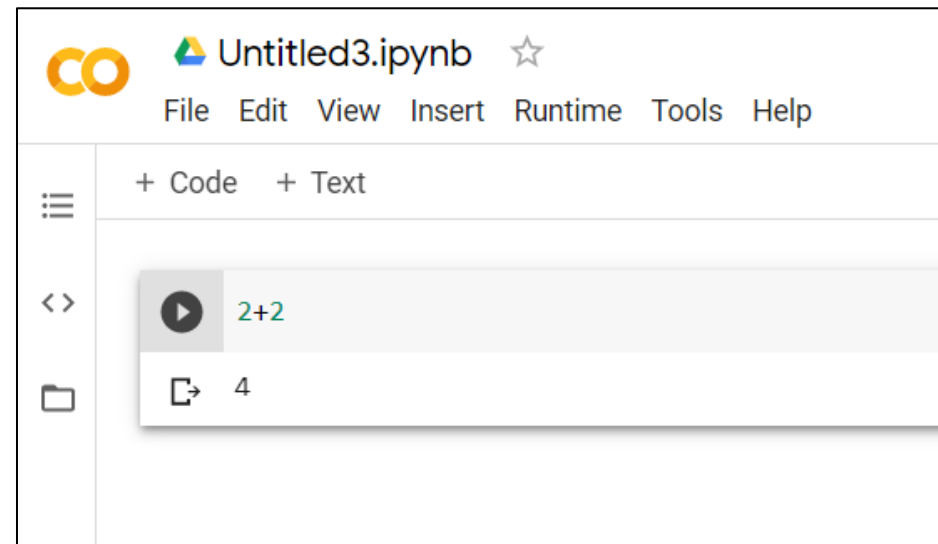
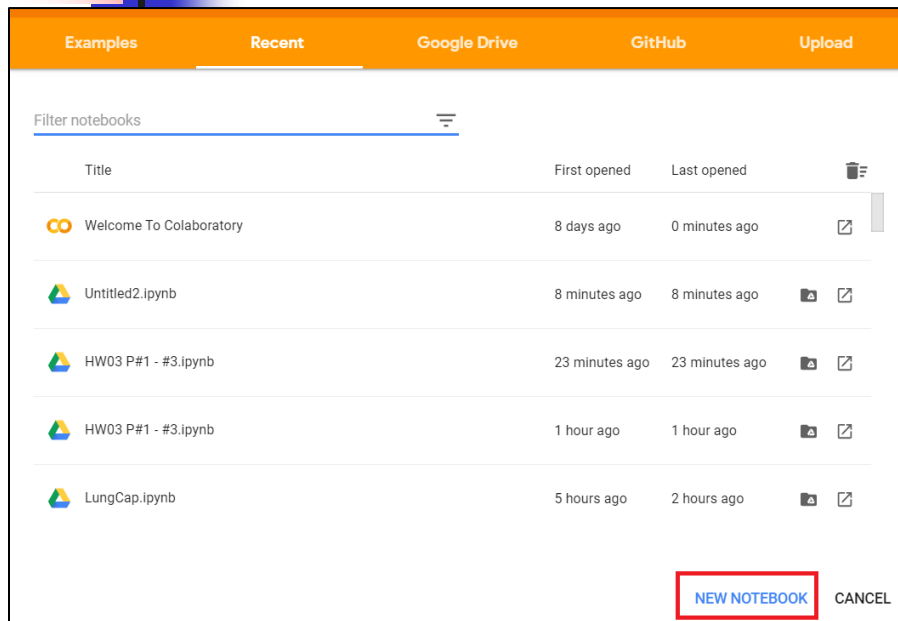
Title	First opened	Last opened	
Welcome To Colaboratory	8 days ago	0 minutes ago	
Untitled2.ipynb	8 minutes ago	8 minutes ago	



Select Jupyter File

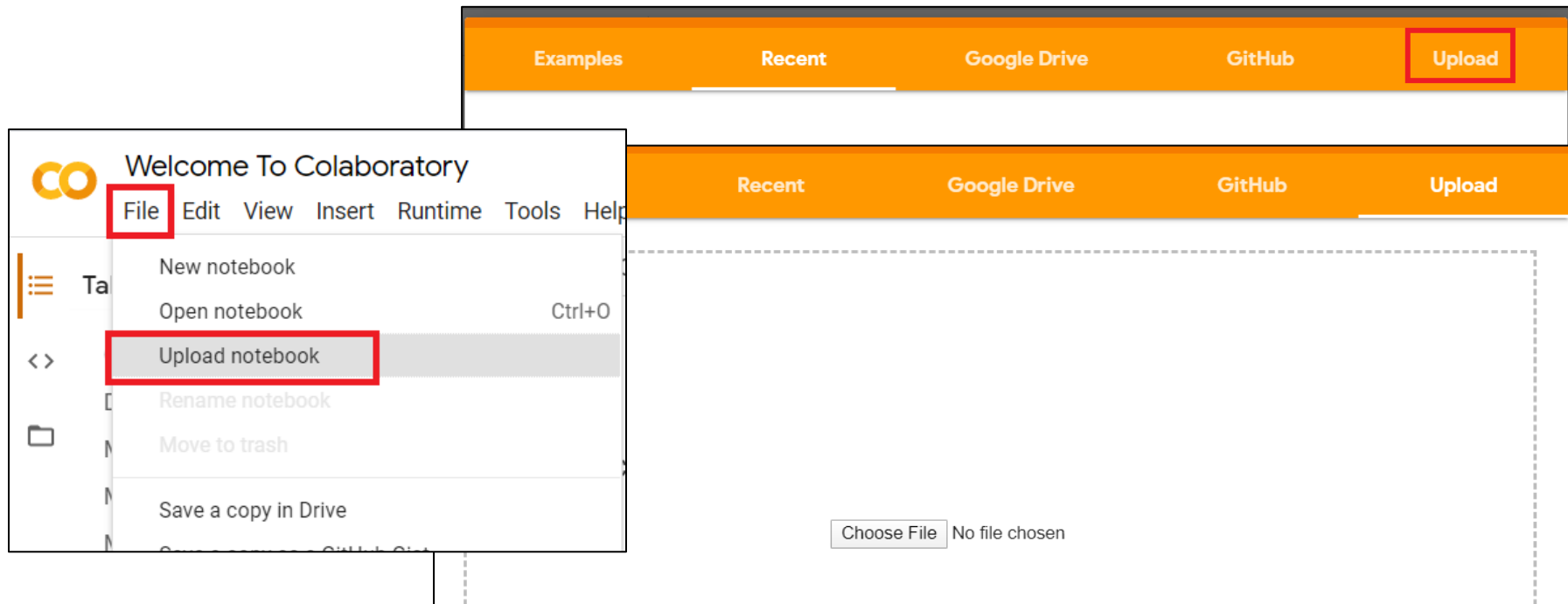
1. Start a New Jupyter File
2. Select a Jupyter file from your Computer
3. Select a Jupyter file from Google Drive

1. Start a New Jupyter File Select: New Notebook

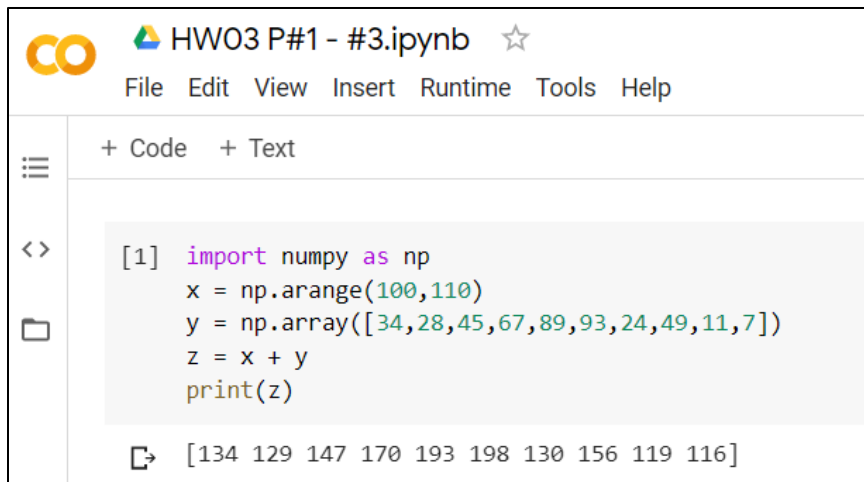


2. Select a Jupyter file from your Computer

- Upload: Select a file from your computer



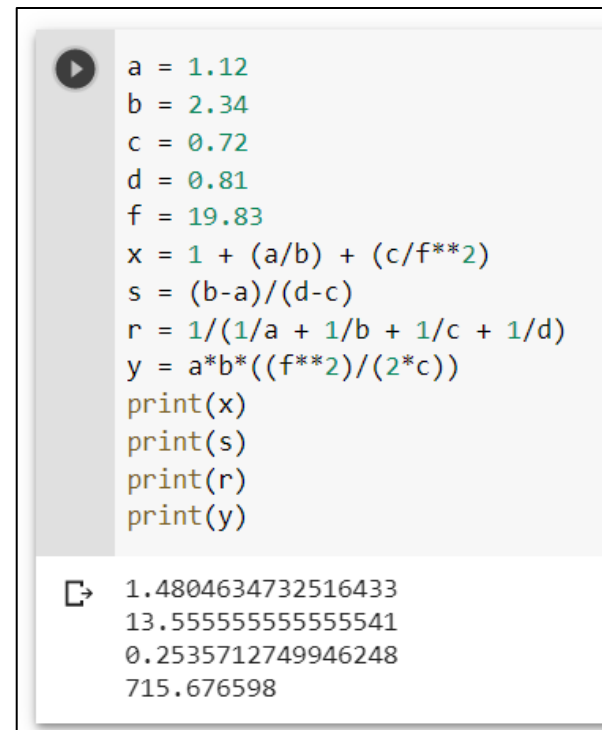
Run Python Code in Colab



The image shows a Google Colab notebook interface. At the top, there's a logo and the title "HW03 P#1 - #3.ipynb" with a star icon. Below the title is a menu bar with "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". On the left side, there's a sidebar with a hamburger menu, a code icon, and a folder icon. The main area has a toolbar with "+ Code" and "+ Text". Below the toolbar is a code cell with the following code:

```
[1] import numpy as np
x = np.arange(100,110)
y = np.array([34,28,45,67,89,93,24,49,11,7])
z = x + y
print(z)
```

Below the code cell, there's an output icon and the output array: [134 129 147 170 193 198 130 156 119 116].



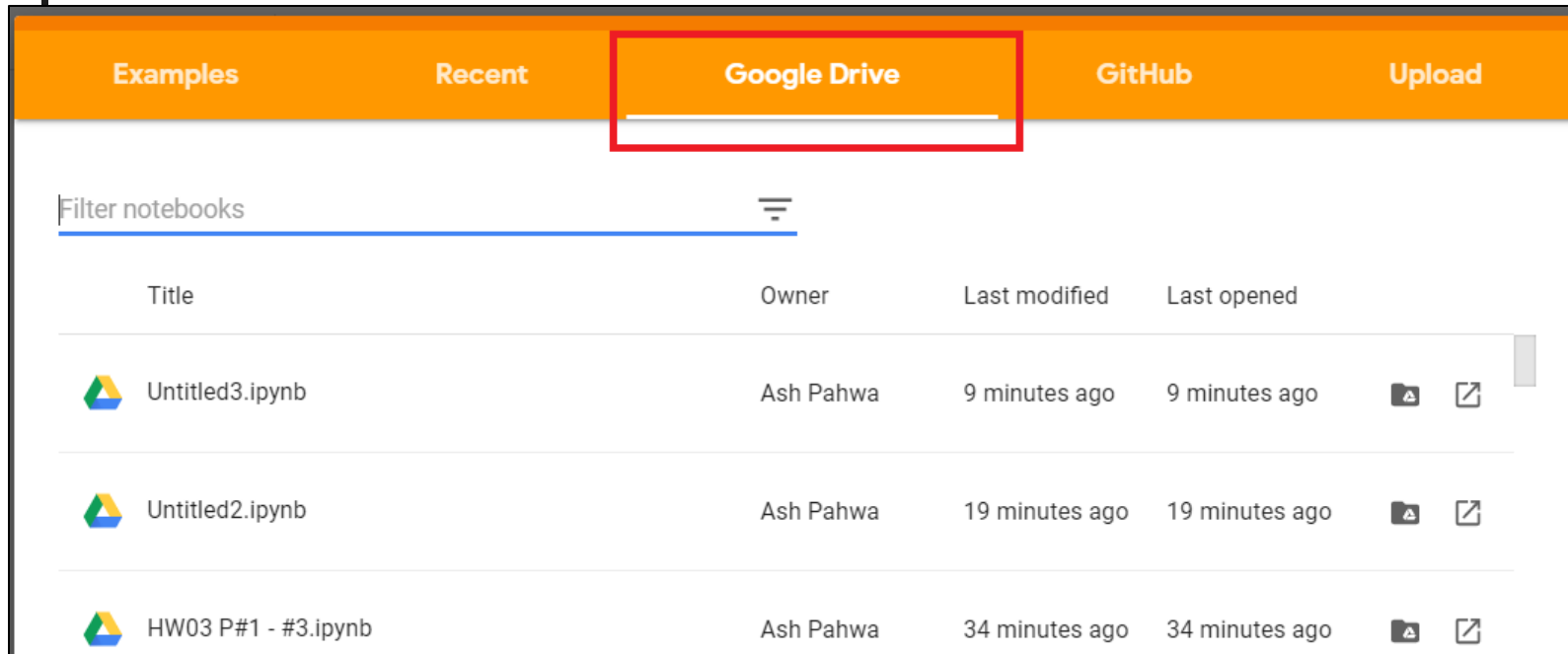
The image shows a Google Colab notebook interface. At the top, there's a play button icon. Below it is a code cell with the following code:

```
a = 1.12
b = 2.34
c = 0.72
d = 0.81
f = 19.83
x = 1 + (a/b) + (c/f**2)
s = (b-a)/(d-c)
r = 1/(1/a + 1/b + 1/c + 1/d)
y = a*b*((f**2)/(2*c))
print(x)
print(s)
print(r)
print(y)
```

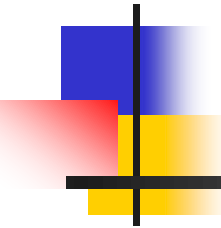
Below the code cell, there's an output icon and the output values:

```
1.4804634732516433
13.555555555555541
0.2535712749946248
715.676598
```

3. Select a Jupyter file from Google Drive



Examples					Recent		Google Drive		GitHub		Upload	
Filter notebooks												
Title					Owner		Last modified		Last opened			
 Untitled3.ipynb					Ash Pahwa		9 minutes ago		9 minutes ago		 	
 Untitled2.ipynb					Ash Pahwa		19 minutes ago		19 minutes ago		 	
 HW03 P#1 - #3.ipynb					Ash Pahwa		34 minutes ago		34 minutes ago		 	

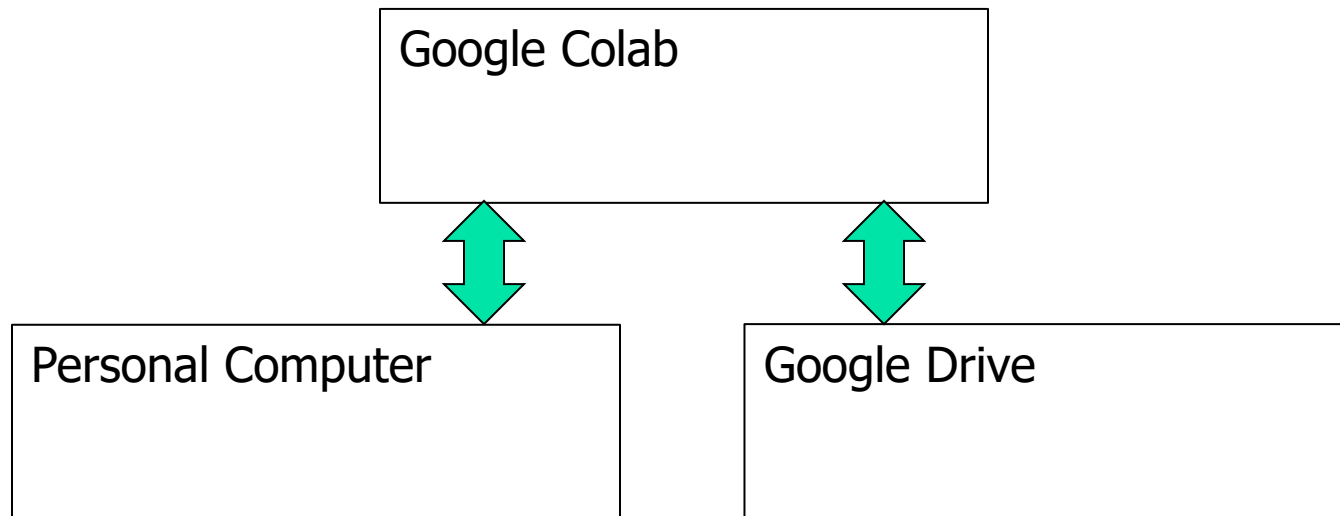


Data IO in Colab

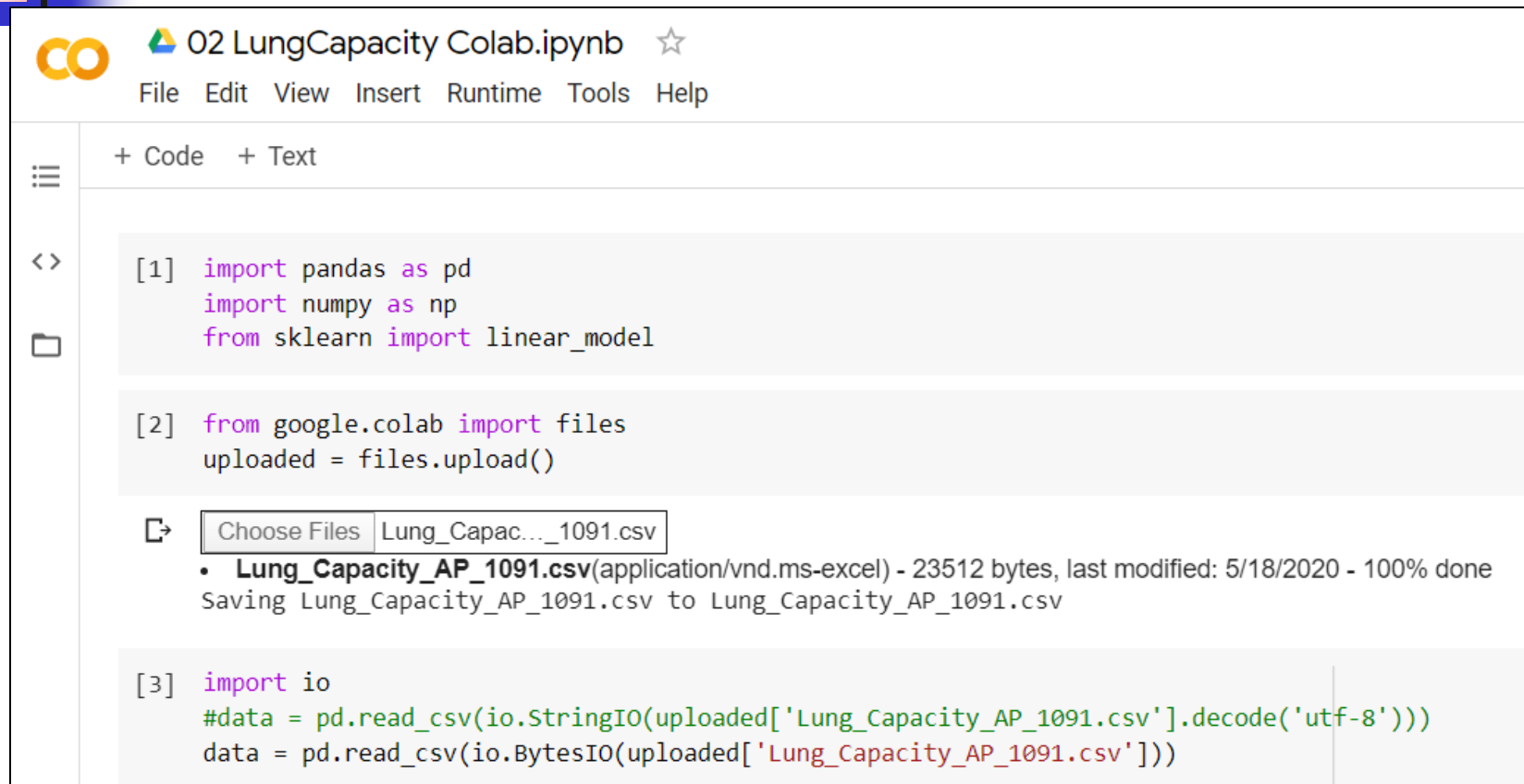
Data (I/O) File in Colab

Load Data, Save Data

- From Personal Computer
- From Google Drive



Loading Data From Personal Computer



The image shows a Google Colab notebook titled "02 LungCapacity Colab.ipynb". The interface includes a menu bar with "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". On the left, there are icons for a menu, code execution, and a file explorer. The main area contains three code cells. The first cell imports pandas as pd, numpy as np, and linear_model from sklearn. The second cell imports files from google.colab and calls files.upload(). Below this, a file selection dialog shows "Lung_Capac..._1091.csv" selected. A message indicates the file is saved. The third cell imports io and uses it to read the uploaded CSV file into a pandas DataFrame.

```
[1] import pandas as pd
import numpy as np
from sklearn import linear_model

[2] from google.colab import files
uploaded = files.upload()

Choose Files Lung_Capac..._1091.csv
• Lung_Capacity_AP_1091.csv(application/vnd.ms-excel) - 23512 bytes, last modified: 5/18/2020 - 100% done
Saving Lung_Capacity_AP_1091.csv to Lung_Capacity_AP_1091.csv

[3] import io
#data = pd.read_csv(io.StringIO(uploaded['Lung_Capacity_AP_1091.csv'].decode('utf-8')))
data = pd.read_csv(io.BytesIO(uploaded['Lung_Capacity_AP_1091.csv']))
```

Loading Data From Personal Computer

```
[5] import io
     data = pd.read_csv(io.StringIO(uploaded['Lung_Capacity_AP_1091.csv'].decode('utf-8')))
```

```
▶ data = pd.read_csv("Lung_Capacity_AP_1091.csv")

print(data.head())
print(data.tail())
data.shape
```

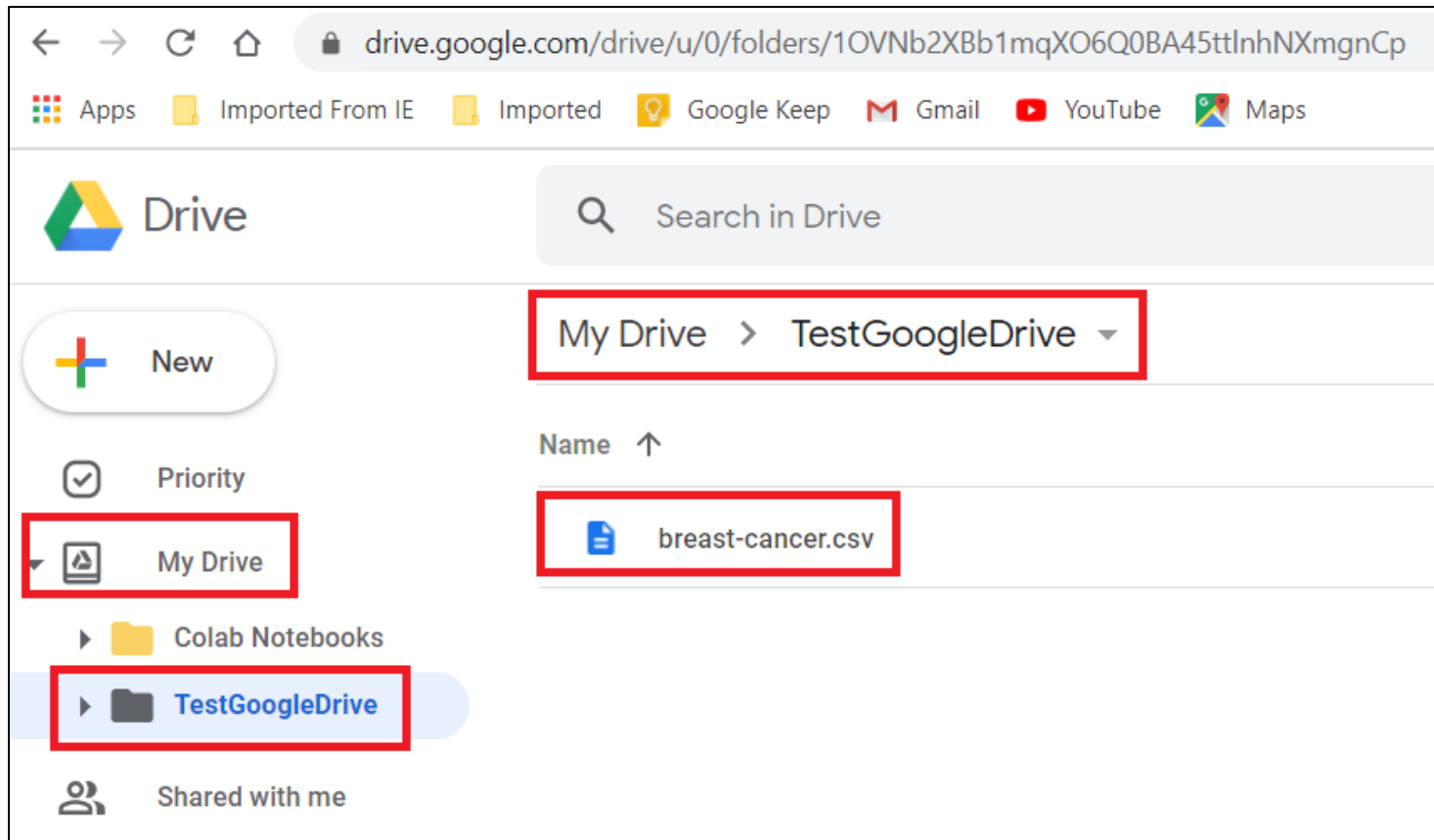
```
↗
```

	Age	LungCap	Height	Gender	Smoke
0	9	3.124	57.0	female	no
1	8	3.172	67.5	female	no
2	7	3.160	54.5	female	no
3	9	2.674	53.0	male	no
4	9	3.685	57.0	male	no

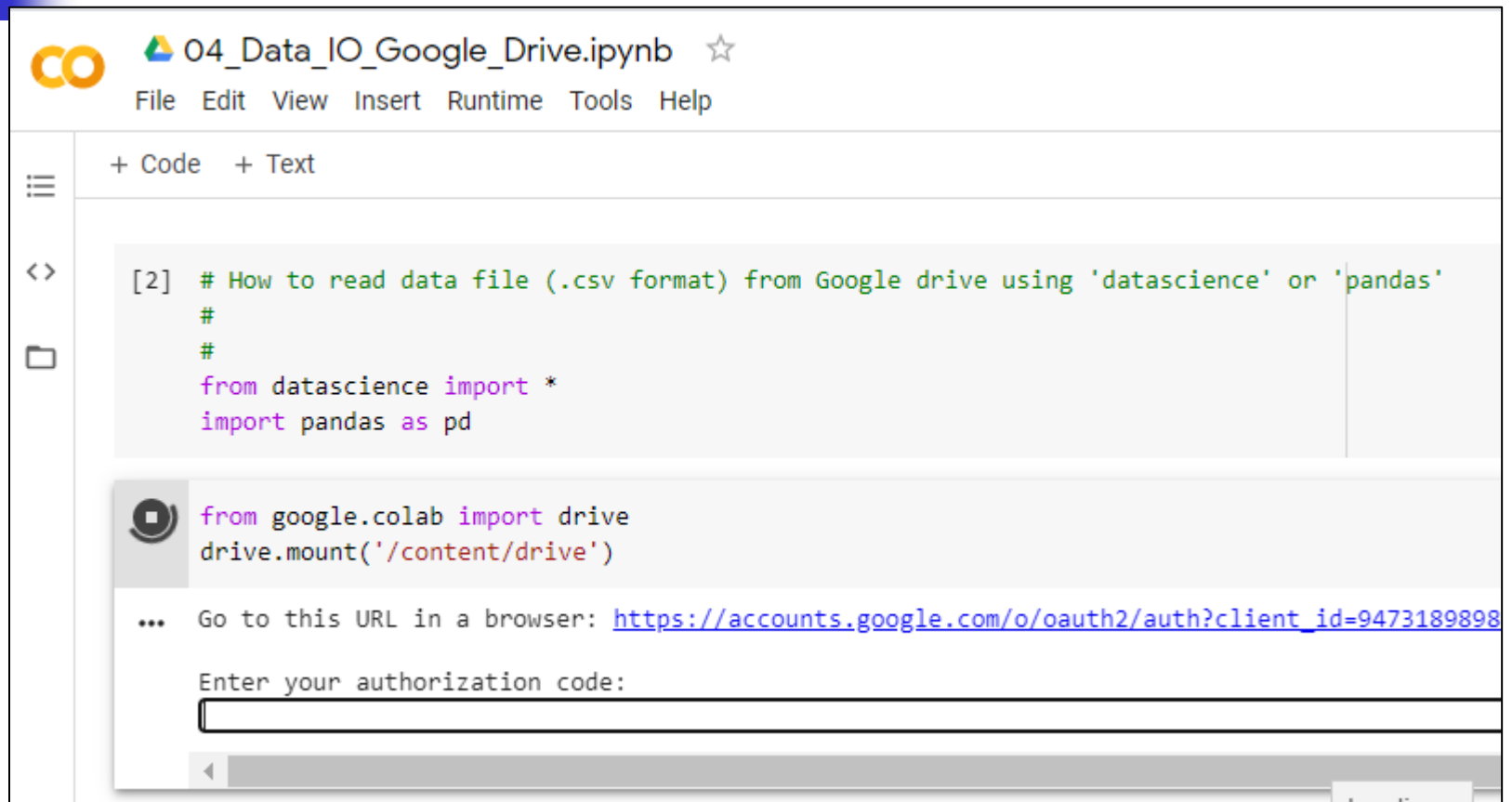
	Age	LungCap	Height	Gender	Smoke
1086	12	8.557	65.5	female	no
1087	13	10.696	70.5	male	no
1088	10	6.310	62.0	male	no
1089	12	8.023	65.5	female	no
1090	10	7.270	65.0	male	no

(1091, 5)

Loading Data From Google Drive



Loading Data From Google Drive



The image shows a Google Colab notebook titled "04_Data_IO_Google_Drive.ipynb". The interface includes a menu bar with "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". Below the menu bar, there are tabs for "+ Code" and "+ Text". The notebook contains two code cells. The first cell is a comment: "[2] # How to read data file (.csv format) from Google drive using 'datascience' or 'pandas'". The second cell contains the following code:

```
from datascience import *  
import pandas as pd  
  
from google.colab import drive  
drive.mount('/content/drive')
```

 Below the code cells, there is a message: "... Go to this URL in a browser: https://accounts.google.com/o/oauth2/auth?client_id=9473189898". Below the URL, there is a text input field labeled "Enter your authorization code:".

```
co 04_Data_IO_Google_Drive.ipynb ☆  
File Edit View Insert Runtime Tools Help  
  
+ Code + Text  
  
[2] # How to read data file (.csv format) from Google drive using 'datascience' or 'pandas'  
#  
#  
from datascience import *  
import pandas as pd  
  
from google.colab import drive  
drive.mount('/content/drive')  
  
... Go to this URL in a browser: https://accounts.google.com/o/oauth2/auth?client\_id=9473189898  
  
Enter your authorization code:  

```

Give Permission to Access Google Drive

Copy the Authorization Code

Google Drive File Stream wants to access your Google Account

 ash@ashpahwa.com

This will allow **Google Drive File Stream** to:

-  See, edit, create, and delete all of your Google Drive files 
-  View the photos, videos and albums in your Google Photos 
- ☒ View Google people information such as profiles and contacts 
- ☒ See, edit, create, and delete any of your Google Drive documents 

Make sure you trust Google Drive File Stream

You may be sharing sensitive info with this site or app. Learn about how Google Drive File Stream will handle your data by reviewing its [terms of service](#) and [privacy policies](#). You can always see or remove access in your [Google Account](#).

[Learn about the risks](#)

[Cancel](#) [Allow](#)

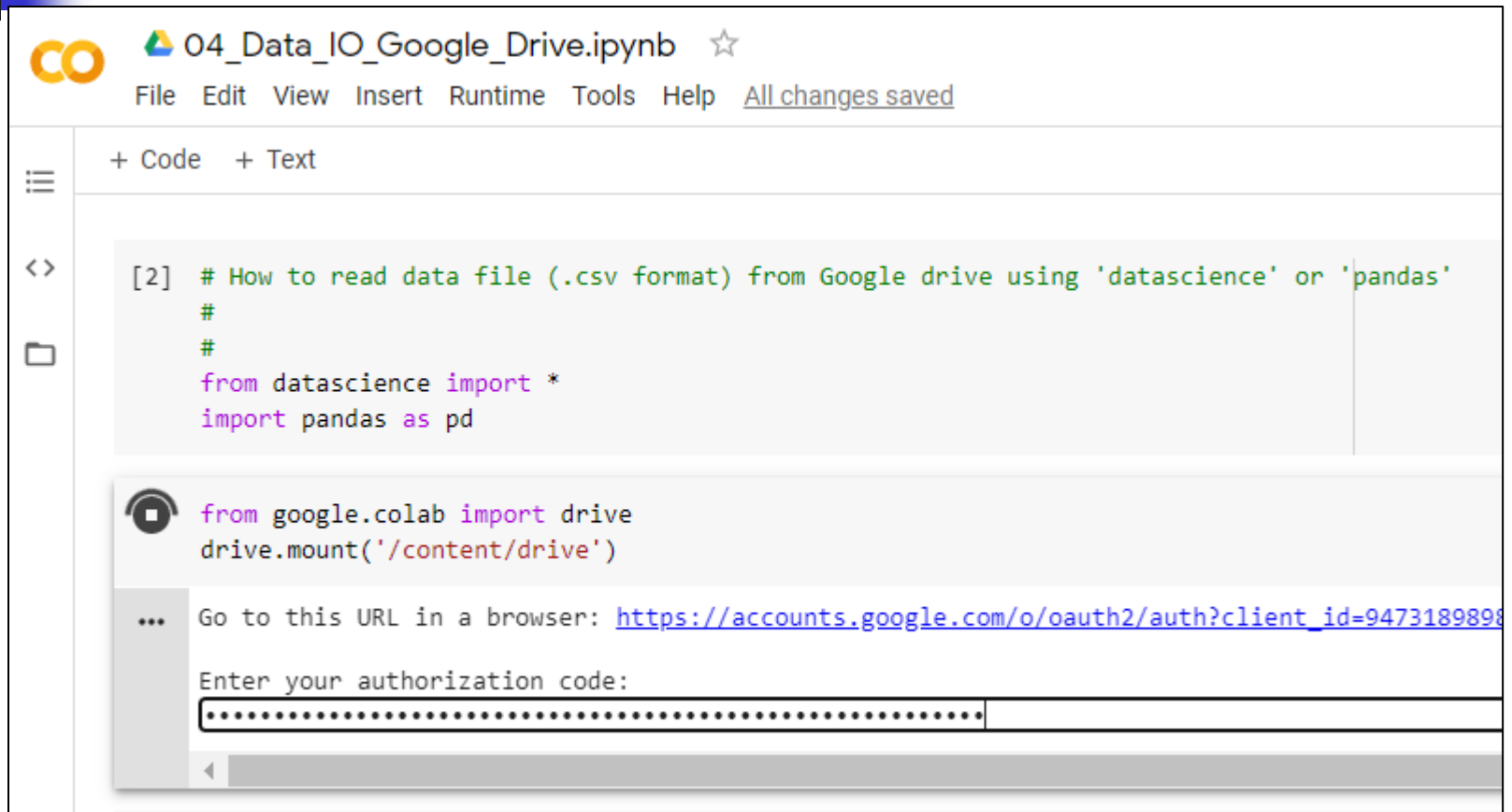


Sign in

Please copy this code, switch to your application and paste it there:

**4/1wEiqEXKF-
GiA838x9PVFmdygyqYp6tebocAeekB23sw2i2x2Sxc8Qs** 

Paste the Authorization Code

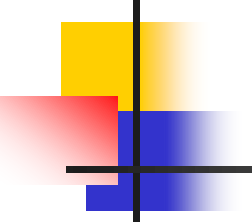


The image shows a Google Colab notebook titled "04_Data_IO_Google_Drive.ipynb". The notebook has a menu bar with "File", "Edit", "View", "Insert", "Runtime", "Tools", "Help", and "All changes saved". Below the menu bar, there are tabs for "+ Code" and "+ Text". The notebook contains two code cells. The first cell is a comment: "[2] # How to read data file (.csv format) from Google drive using 'datascience' or 'pandas'". The second cell contains the following code:

```
from datascience import *
import pandas as pd
```

Below the code cells, there is a prompt to enter an authorization code. The prompt says: "Go to this URL in a browser: https://accounts.google.com/o/oauth2/auth?client_id=9473189898". Below the URL, there is a text input field with the label "Enter your authorization code:".

Read File From Google Drive



04_Data_IO_Google_Drive.ipynb ☆

File Edit View Insert Runtime Tools Help

+ Code + Text

```
[3] from google.colab import drive
drive.mount('/content/drive')
```

Go to this URL in a browser: https://accounts.google.com/o/oauth2/auth?client_id=947318989803-6bn6qk8qdgf4r

Enter your authorization code:
.....
Mounted at /content/drive

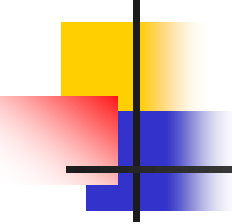
```
patients = Table.read_table('/content/drive/My Drive/TestGoogleDrive/breast-cancer.csv').drop('ID')
patients.show(3)
```

Clump Thickness	Uniformity of Cell Size	Uniformity of Cell Shape	Marginal Adhesion	Single Epithelial C
5	1	1	1	
5	4	4	5	
3	1	1	1	

... (680 rows omitted)

```
[ ] data = pd.read_csv('/content/drive/My Drive/TestGoogleDrive/breast-cancer.csv')
data.head
```

ID	Clump Thickness	...	Mitoses	Class
0 1000025	5	...	1	0
1 1002945	5	...	1	0
2 1015425	3	...	1	0
3 1016277	6	...	1	0
4 1017023	4	...	1	0
...	...	...	...	...
678 776715	3	...	1	0
679 841769	2	...	1	0



Summary

- Colab
 - What is Colab?
 - Cost of Colab
 - Benefits and Drawbacks of Colab
 - How to Use Colab
 - Data IO in Colab